



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa licencjacka

MIĘDZYPLATFORMOWA APLIKACJA MOBILNA DO NAWIGACJI WEWNĘTRZNEJ W BUDYNKACH

Mikołaj Nowak, 151813

Promotor
prof. dr hab. inż. Piotr Formanowicz

POZNAŃ 2025

Spis treści

1	Wstęp	1
1.1	Wprowadzenie	1
1.2	Cel i zakres pracy	1
2	Podstawowe pojęcia	3
3	Przegląd literatury	5
3.1	Wprowadzenie	5
3.2	Podejścia szczegółowe	5
3.3	Prace przeglądowe	6
3.4	Obecnie istniejące systemy	7
4	Materiały i metody	8
4.1	Wprowadzenie	8
4.2	Segmentacja obrazu	8
4.3	Klasyfikacja gdzie wykonano zdjęcie	9
4.4	Detekcja obiektów charakterystycznych	9
4.5	Rozumienie tekstu na tablicach	10
4.6	Algorytmy wyznaczania najlepszej ścieżki	10
4.7	Śledzenie przemieszczenia	10
5	Narzędzia programistyczne	11
5.1	Wprowadzenie	11
5.2	C++	11
5.3	CMake	11
5.4	Djinni	12
5.5	ONNX	13
5.6	ONNX Runtime	14
6	Międzyplatformowość	16
6.1	Wprowadzenie	16
6.2	Moduł IndoorNavigationService	16
7	Początkowa lokalizacja i pozycjonowanie	20
7.1	Wprowadzenie	20
7.2	Mapowanie budynków	20
7.3	Rozpoznawanie obiektów na planie budynku	21
7.4	Klasyfikacja obiektów	22
7.5	Punkty preferowane	23

7.6	Klasyfikacja, gdzie zostało wykonane zdjęcie	24
7.7	Detekcja obiektów charakterystycznych	25
7.8	Lokalizacja w oparciu o pozycję obiektów charakterystycznych	26
7.8.1	Analiza punktów charakterystycznych wykrytych na różnych piętrach	29
7.9	Rozumienie tekstu na tablicach	29
7.9.1	Algorytm Levenshteina z uwzględnieniem błędów końcowych	30
7.10	Dane treningowe	31
8	Śledzenie przemieszczenia	33
8.1	Wprowadzenie	33
8.2	Algorytmy wyznaczania najlepszej ścieżki	33
8.2.1	Trasa na jednej kondygnacji	33
8.2.2	Trasa po wielu kondygnacjach	34
8.3	Parametry algorytmu detekcji kroków	35
8.4	Algorytm detekcji kroków	35
8.5	Analiza liczby przeszkód pomiędzy 2 punktami	36
8.5.1	Różne piętra ($z_1 \neq z_2$)	36
8.5.2	To samo piętro ($z_1 = z_2$)	37
8.6	Wykrywanie zmiany piętra niezależnie od krokomierza	38
8.7	Korekcja kumulujących się błędów	39
8.7.1	Mechanizm wykrywania najbliższych sąsiadów	40
8.7.2	Implementacja algorytmu ICP z SVD	41
9	Sposób Użytkowania Aplikacji	42
9.1	Wprowadzenie	42
9.1.1	Wybór Budynku i Ustawienie Preferencji	42
9.1.2	Przeszukiwanie Listy Punktów Docelowych	42
9.1.3	Lokalizacja użytkownika oraz podążanie do celu	42
10	Zakończenie	46
	Literatura	47

Rozdział 1

Wstęp

1.1 Wprowadzenie

Zdolność lokalizowania użytkownika w przestrzeni była od dawna pożądaną umiejętnością, szczególnie od czasu rozpoczęcia prac nad systemem GPS w 1973 roku [1]. System ten, uznawany obecnie za wystarczająco precyzyjny w większości zastosowań zewnętrznych, napotyka jednak istotne trudności w przestrzeniach zamkniętych. Zakłócenia sygnału GPS wewnątrz budynków, spowodowane tłumieniem i odbijaniem fal, uniemożliwiają efektywne wykorzystanie tej technologii w takich warunkach [2].

Tradycyjne rozwiązania stosowane w budynkach, takie jak punkty i tablice informacyjne, pozwalają użytkownikom odnaleźć swoją drogę, lecz są obciążone istotnymi ograniczeniami. Wymagają one, między innymi, samodzielnego zapamiętania trasy, co bywa problematyczne w dużych, złożonych przestrzeniach. Dodatkowo, tablice informacyjne mogą być niezrozumiałe dla osób nieznających lokalnego języka, a ich instalacja i utrzymanie generują koszty dla zarządców budynków.

Na przestrzeni lat rozwijano różne technologie lokalizacji wewnętrznej, w tym systemy oparte na sygnałach radiowych, Bluetooth, Wi-Fi, a także na danych wizualnych z kamer. Każde z tych rozwiązań ma swoje zalety, ale również wady, takie jak ograniczona skalowalność, koszty implementacji czy złożoność techniczna.

Proponowane w niniejszej pracy rozwiązanie bazuje na smartfonach, które są powszechnie dostępne i wyposażone w szereg czujników, takich jak akcelerometry, żyroskopy, magnetometry czy kamery. Rozwiązanie to minimalizuje konieczność wprowadzania dodatkowych infrastruktur w budynkach, co czyni je bardziej skalowalnym i ekonomicznym.

1.2 Cel i zakres pracy

Celem pracy jest opracowanie systemu informatycznego, który, działając na dowolnym urządzeniu mobilnym niezależnie od systemu operacyjnego, umożliwi dokładną lokalizację użytkownika w wybranym budynku. System śledzi jego przemieszczenie i pomaga dotrzeć do wskazanego punktu docelowego.

Podstawowymi źródłami wykorzystanymi w pracy były publikacje naukowe oraz materiały konferencyjne, w tym prace dotyczące algorytmów lokalizacji w przestrzeniach zamkniętych. Ważnym elementem były także dokumentacje techniczne producentów urządzeń mobilnych, szczególnie w zakresie funkcji wbudowanych sensorów, między innymi takich jak akcelerometry, żyroskopy i kamery. Opracowanie systemu opierało się również na danych empirycznych zebranych podczas testów w rzeczywistych warunkach.

Struktura pracy jest następująca. Rozdział 2 zawiera przegląd literatury dotyczący metod początkowego wyznaczania oraz ciągłej aktualizacji położenia użytkownika w dowolnym układzie współrzędnych. Szczególny nacisk położono na lokalizację wewnątrz budynków. Analizowane są dostępne rozwiązania technologiczne oraz ich ograniczenia, a także potencjalne możliwości ich adaptacji do specyfiki nawigacji wewnętrznej. Rozdział 3 opisuje proces przekształcania planów budynków na format zrozumiały zarówno dla proponowanego systemu, jak i dla użytkownika końcowego. Przedstawiono metody umożliwiające intuicyjną interpretację map przez człowieka w kontekście rzeczywistego otoczenia. Szczególną uwagę poświęcono dokładności odwzorowania i użyteczności wizualnej, która ma kluczowe znaczenie w budowaniu zaufania do systemu. W rozdziale 4 omówiono proces gromadzenia i przetwarzania danych treningowych, które zostały wykorzystane do uczenia modeli sztucznej inteligencji oraz algorytmów uczenia maszynowego. Przedstawiono narzędzia i techniki stosowane podczas przygotowywania zbiorów danych oraz opisano ich znaczenie dla poprawności działania systemu. Rozdział 5 jest poświęcony metodom wyznaczania początkowej pozycji użytkownika. Opisano wykorzystanie danych z kamery oraz podstawowych czujników wbudowanych w smartfony, takich jak akcelerometr, żyroskop czy magnetometr. Szczegółowo przedstawiono sposób określania, czy użytkownik znajduje się wewnątrz budynku, oraz metodologię dokładnego ustalania pozycji w przestrzeni wewnętrznej. W rozdziale 6 zaprezentowano algorytmy wyznaczania optymalnej ścieżki przemieszczania się użytkownika. Omówiono mechanizmy uwzględniania indywidualnych preferencji użytkownika, takich jak wybór trasy o najmniejszym dystansie, unikającej schodów lub dostosowanej do osób o ograniczonej mobilności. Uwzględniono również wpływ specyficznych warunków, takich jak dostępność tras. Rozdział 7 skupia się na śledzeniu przemieszczenia użytkownika w czasie rzeczywistym. Przedstawiono zastosowanie czujników, takich jak akcelerometr, żyroskop oraz barometr, do aktualizacji pozycji użytkownika w układzie współrzędnych. Omówiono również sposoby kompensacji potencjalnych błędów wynikających z dryftu czujników oraz zakłóceń zewnętrznych. W rozdziale 8 opisano, w jaki sposób osiągnięto wieloplatformowość aplikacji. Przedstawiono podejście umożliwiające minimalizację różnic w implementacji między różnymi systemami operacyjnymi oraz architekturami procesorów. Szczególną uwagę poświęcono technologiom umożliwiającym utrzymanie jednolitej bazy kodu oraz mechanizmom dostosowującym aplikację do specyficznych wymagań poszczególnych platform. Rozdział 9 stanowi podsumowanie pracy.

Rozdział 2

Podstawowe pojęcia

Współczesne technologie informatyczne w znacznym stopniu wpłynęły na sposób, w jaki ludzie korzystają z urządzeń mobilnych w codziennym życiu. Użytkownik, czyli każda osoba posiadająca smartfona, może korzystać z zaawansowanych funkcji takich urządzeń, które jeszcze kilka dekad temu były jedynie przedmiotem badań naukowych. Smartfony, wyposażone w różnorodne czujniki i komponenty, stały się narzędziami umożliwiającymi wykonywanie złożonych zadań, takich jak nawigacja, śledzenie aktywności fizycznej czy identyfikacja obiektów w przestrzeni.

System operacyjny to oprogramowanie, które zarządza działaniem urządzenia mobilnego, umożliwiając użytkownikowi korzystanie z aplikacji oraz funkcji sprzętowych. Do najpopularniejszych systemów operacyjnych należą Android oraz iOS. Android, opracowany przez firmę Google, jest systemem otwartym, co oznacza, że może być modyfikowany i dostosowywany przez różnych producentów urządzeń. Z kolei iOS, stworzony przez Apple, charakteryzuje się większym zamknięciem ekosystemu, co zapewnia lepszą integrację z innymi produktami tej firmy, ale ogranicza możliwość modyfikacji przez użytkownika.

Jednym z kluczowych wymagań współczesnych aplikacji mobilnych jest międzyplatformowość, czyli zdolność oprogramowania do działania na różnych systemach operacyjnych i urządzeniach bez konieczności wprowadzania istotnych zmian w kodzie źródłowym. Rozwiązania międzyplatformowe umożliwiają programistom tworzenie aplikacji, które działają zarówno na Androidzie, jak i iOS, co znacząco obniża koszty i czas potrzebny na ich rozwój oraz utrzymanie.

Współczesne smartfony są wyposażone w szereg czujników, które odgrywają istotną rolę w systemach lokalizacyjnych. Akcelerometr mierzy przyspieszenie liniowe urządzenia, umożliwiając wykrywanie ruchu, takich jak kroki użytkownika. Żyroskop dostarcza informacji o obrotach urządzenia, co pozwala na precyzyjne określenie jego orientacji w przestrzeni. Magnetometr mierzy natężenie pola magnetycznego, co pozwala na wyznaczenie kierunku północnego i pełni funkcję cyfrowego kompasu. Barometr z kolei rejestruje zmiany ciśnienia atmosferycznego, co może być wykorzystane do określenia wysokości nad poziomem morza. Lidar, czyli technologia wykrywania i pomiaru odległości za pomocą światła laserowego, umożliwia tworzenie dokładnych trójwymiarowych map otoczenia, co znajduje zastosowanie w systemach nawigacyjnych oraz autonomicznych pojazdach.

Aby otrzymać sprawnie działający system nawigacyjny, który potrafi lokalizować użytkownika w przestrzeni, a następnie śledzić jego przemieszczenie potrzebny jest algorytm, czyli zestaw kroków lub instrukcji, które prowadzą do rozwiązania określonego problemu. W kontekście systemów lokalizacyjnych algorytmy są kluczowe dla przetwarzania danych z czujników i wyznaczania optymalnej ścieżki do celu. Wraz z rozwojem sztucznej inteligencji algorytmy stają się coraz bardziej zaawansowane, co umożliwia automatyzację zadań wymagających wcześniej interwencji człowieka.

Sztuczna inteligencja umożliwia tworzenie automatycznych systemów zdolnych do podejmowania decyzji, rozwiązywania problemów czy uczenia się na podstawie danych bez ingerencji użytkownika w ich działanie. Pozwala między innymi na rozpoznawanie i lokalizowanie określonych elementów w obrazach cyfrowych. W systemach lokalizacyjnych może to być wykorzystane do identyfikacji charakterystycznych punktów, takich jak drzwi, schody czy tablice informacyjne. Podobnym wyzwaniem jest czytanie tekstu z obrazów, znane jako OCR (ang. Optical Character Recognition), które pozwala na zamianę informacji wizualnej w postać cyfrową.

Aby sztuczna inteligencja potrafiła ze znaczącą dokładnością rozpoznawac teksty i obiekty na zdjęciach, musi zostać jej dostarczony zbiór treningowy, czyli zestaw danych składający się z przykładów zawierających dane wejściowe i odpowiadające im oczekiwane wyniki. Jakość i różnorodność danych w zbiorze treningowym mają kluczowe znaczenie dla skuteczności systemu, ponieważ umożliwiają mu nauczanie się prawidłowych wzorców i reagowania na nowe sytuacje.

Powyższe pojęcia i technologie stanowią podstawę dla rozwoju systemów lokalizacyjnych i zostaną szczegółowo omówione w kolejnych rozdziałach pracy.

Rozdział 3

Przegląd literatury

3.1 Wprowadzenie

Literatura dotycząca systemów nawigacji wewnętrznej dzieli się na dwie główne kategorie: prace skupiające się na szczegółowych przykładach technologicznych oraz prace przeglądowe, które podsumowują dotychczasowe osiągnięcia i proponują przyszłe kierunki rozwoju. W tym rozdziale omówione zostaną prace, które prezentują konkretne podejścia i przykłady rozwiązań, które doprowadziły do powstania działających systemów nawigacji wewnętrznej. W poniższej pracy skupiono się na tych badaniach, które miały bezpośredni wpływ na tworzenie funkcjonalnych aplikacji i rozwiązań nawigacyjnych w rzeczywistych środowiskach, takich jak centra handlowe, szpitale czy lotniska. Ponadto, omówiono prace przeglądowe, które miały na celu podsumowanie dotychczasowego stanu wiedzy i klasyfikację różnych metod oraz technologii wykorzystywanych w tych systemach.

3.2 Podejścia szczegółowe

W pracy Grinyak i współautorów [3] szczegółowo opisano system nawigacji oparty na wykorzystaniu sygnałów Bluetooth Low Energy. System korzysta z gęsto rozmieszczonych nadajników BLE, które przesyłają sygnały do urządzeń mobilnych, pozwalając na oszacowanie ich pozycji. Główną zaletą tego podejścia jest stosunkowo niska cena i możliwość uzyskania dokładności rzędu 1-2 metrów. Jednak autorzy wskazują na wady, takie jak konieczność utrzymania dużej liczby nadajników w infrastrukturze oraz zakłócenia sygnału powodowane przez przeszkody architektoniczne i urządzenia elektromagnetyczne.

W pracy Li i współpracowników [4] ponownie omówiono wykorzystanie sygnałów BLE w systemach lokalizacyjnych. Przeprowadzone badania pokazały, że dokładność systemu może być wysoka, ale wymaga to precyzyjnego rozmieszczenia nadajników, co zwiększa koszty instalacji i utrzymania. Dodatkowo autorzy powtórzyli wnioski z poprzednio wspomnianej pracy, że sygnały BLE są podatne na zakłócenia w środowiskach z dużą liczbą przeszkód.

Rosen [5] przedstawił system oparty na akcelerometrach i żyroskopach, który wykorzystuje technologię inercyjną INS. W pracy skupiono się na zastosowaniu w nawigacji dla osób niewidomych. INS umożliwia dokładne śledzenie pozycji w krótkim okresie czasu, co jest szczególnie przydatne w środowiskach, gdzie sygnały zewnętrzne, takie jak GPS, są niedostępne. Jednak Rosen wskazuje na problem dryfu, który powoduje narastające błędy lokalizacji w dłuższych okresach użytkowania.

W pracy Brinthakumari [6] zaproponowano system oparty na rozszerzonej rzeczywistości (AR), który wykorzystuje kamery urządzeń mobilnych do lokalizacji użytkownika i generowania wskazówek nawigacyjnych w czasie rzeczywistym. System integruje dane wizualne z przestrzennymi, co

pozwała na precyzyjne odwzorowanie otoczenia. Zalety tego rozwiązania to niski koszt implementacji (przy założeniu wykorzystania istniejących urządzeń mobilnych) oraz intuicyjny interfejs użytkownika. Główne wady to wrażliwość systemu na zmienne warunki oświetleniowe oraz wysoka moc obliczeniowa wymagana do przetwarzania obrazów.

Khan i współautorzy [7] opisali podejście oparte na wizji komputerowej, gdzie system nawigacyjny również analizuje dane z kamer, aby identyfikować pozycję użytkownika względem znanych punktów odniesienia. Metoda ta jest dokładna i eliminuje potrzebę dedykowanej infrastruktury, co obniża koszty wdrożenia. Jednak autorzy podkreślają, że efektywność systemu zależy od jakości obrazu oraz dostępności wysokiej mocy obliczeniowej na urządzeniu użytkownika.

Chiang i współpracownicy [8] przeanalizowali integrację technologii INS/GNSS/V-SLAM w kontekście nawigacji pojazdów lądowych w środowiskach, gdzie sygnał GNSS jest ograniczony. Zastosowanie tej technologii umożliwia precyzyjne śledzenie pozycji, łącząc dane z czujników umieszczonych w smartfonach. Badania wykazały, że integracja tych technologii pozwala na znaczne zwiększenie dokładności lokalizacji, szczególnie w obszarach, gdzie klasyczne systemy GNSS mają ograniczoną użyteczność. Praca podkreśla również wyzwania związane z synchronizowaniem danych z różnych sensorów.

Sen [9] zaprezentował metodę integracji punktów orientacyjnych (landmarks) wyodrębnionych z opisów tras użytkowników, korzystając z przetwarzania języka naturalnego (NLP), w modelu nawigacyjnym przestrzeni wewnętrznych. Autor proponuje innowacyjne podejście, które łączy informacje uzyskane z tras użytkowników z danymi o przestrzeni wewnętrznej, co pozwala na efektywne budowanie i aktualizowanie map nawigacyjnych. To rozwiązanie jest szczególnie użyteczne w dynamicznych środowiskach, gdzie tradycyjne metody mapowania mogą być nieefektywne.

Yasar i współautorzy [10] opisali metodę wykorzystania chmur punktów wewnętrznych do planowania ścieżek i nawigacji w sytuacjach kryzysowych, takich jak pożary czy wypadki. W pracy skupiono się na analizie danych przestrzennych uzyskanych z chmur punktów, które umożliwiają tworzenie szczegółowych modeli 3D przestrzeni. Dzięki temu możliwe jest precyzyjne planowanie tras ewakuacyjnych, a także skuteczniejsza nawigacja w trudnych warunkach, gdzie tradycyjne systemy mogą zawodzić. Zaletą tej technologii jest możliwość dostosowania do różnych typów środowisk wewnętrznych, co czyni ją wszechstronną w zastosowaniach ratunkowych.

W pracy Morganti i współpracowników [11] zaprezentowano podejście transdyscyplinarne w nawigacji wewnętrznej, które integruje rzeczywistość mieszaną z chmurą obliczeniową. Badania koncentrują się na wykorzystaniu holograficznych technologii w połączeniu z danymi przestrzennymi, co ma na celu poprawę doświadczeń użytkownika w środowiskach wewnętrznych.

Maurya [12] zaprezentował system nawigacji wewnętrznej, który skupia się na technologii opartej na systemach śledzenia lokalizacji w pomieszczeniach. W artykule autor omawia sposób, w jaki różne technologie mogą współdziałać w tworzeniu dokładnego systemu nawigacji, który integruje dane z urządzeń mobilnych. Zastosowanie takich systemów w różnych przestrzeniach wewnętrznych, jak biura czy centra handlowe, może znacząco poprawić jakość usług nawigacyjnych. Maurya zwraca uwagę na wyzwania związane z integracją różnych technologii oraz potrzebę uzyskania wysokiej precyzji lokalizacji w trudnych warunkach.

3.3 Prace przeglądowe

Tadepalli i współpracownicy [13] dokonali przeglądu innowacyjnych rozwiązań nawigacyjnych opartych na rozszerzonej rzeczywistości. Autorzy podkreślili potencjał tej technologii w lokalizacji wewnętrznej dzięki jej zdolności do wizualizacji otoczenia i integracji z danymi przestrzennymi.

Zwrócono jednak uwagę na ograniczenia związane z zasobami sprzętowymi, które mogą ograniczać jej szerokie zastosowanie.

El-Sheimy i Li [14] również przeanalizowali zalety i wady różnych technologii lokalizacji wewnętrznej, w tym sygnałów Bluetooth, wizji komputerowej oraz podejść inercyjnych. Autorzy podkreślili, że integracja tych technologii pozwala na znaczne zwiększenie precyzji lokalizacji w trudnych warunkach wewnętrznych. Jednak zwrócono uwagę na wyzwania związane z kosztami i złożonością implementacji hybrydowych systemów.

Singh i współpracownicy [15] przedstawili systematyczny przegląd współczesnych systemów lokalizacji wewnętrznej, omawiając różnorodne techniki i algorytmy stosowane w tych systemach. Zwrócili uwagę na zmieniające się wymagania dotyczące dokładności i efektywności, a także na wyzwania związane z implementacją systemów IPS w różnych środowiskach.

Deliyska i współpracownicy [4] przedstawili metody opracowywania systemów nawigacji wewnętrznej, koncentrując się na różnych podejściach, które mogą być wykorzystane do efektywnego i dokładnego śledzenia pozycji w zamkniętych przestrzeniach. Autorzy omawiają zalety oraz wady poszczególnych technologii, takich jak czujniki inercyjne, Bluetooth, Wi-Fi oraz systemy oparte na obrazie. Ich badania pokazują, że wykorzystanie wielu źródeł danych pozwala na uzyskanie bardziej precyzyjnych wyników w trudnych warunkach wewnętrznych.

3.4 Obecnie istniejące systemy

Jednym z najbardziej popularnych systemów nawigacji wewnętrznej są Google Maps [16]. Mimo swojej szerokiej popularności, system ten posiada istotne ograniczenia, zwłaszcza w sytuacjach, gdy użytkownik nie ma dostępu do internetu lub sygnału GPS. W takich warunkach precyzyjna lokalizacja staje się znacznie trudniejsza, a w niektórych przypadkach korzystanie z aplikacji jest w ogóle niemożliwe. Alternatywą dla tworzenia map budynków jest system MapCreator dostępny w systemie Android [17]. Aplikacja ta wymaga jednak wcześniejszego zmapowania budynku, co wiąże się z koniecznością uzyskania zgody administratora obiektu. Ograniczenie to znacząco wydłuża proces wprowadzania nowych budynków do systemu, a dodatkowo rozwiązanie to okazuje się niewystarczająco elastyczne w obliczu zmian w organizacji przestrzeni wewnętrznej obiektu.

Podobne ograniczenia można zaobserwować w przypadku innych systemów, takich jak Situm [18], czy BeMyEyes [19], który został zaprojektowany głównie z myślą o osobach niewidomych. Ponadto, wiele z istniejących rozwiązań nie spełnia podstawowych założeń przyjętych w niniejszej pracy, takich jak na przykład wieloplatformowość. Warto również zauważyć, że firma Apple wprowadza ograniczenia dotyczące ciągłego skanowania sygnałów Wi-Fi w swoich urządzeniach [20], co stanowi istotną przeszkodę w realizacji systemów lokalizacyjnych bazujących na tej technologii.

Według najlepszej wiedzy autora niniejszej pracy, obecnie nie istnieje system, który umożliwiałby dokładną lokalizację wewnętrzną bez konieczności wcześniejszego skanowania budynku lub instalacji nadajników, zwanych beaconami, w kluczowych punktach obiektu. Proponowane rozwiązanie w ramach niniejszej pracy ma na celu wyeliminowanie powyższych ograniczeń, oferując bardziej elastyczne i niezależne podejście do lokalizacji wewnętrznej.

Rozdział 4

Materiały i metody

4.1 Wprowadzenie

W poniższym rozdziale przedstawiono narzędzia, technologie oraz algorytmy, które stanowią fundamenty realizacji systemu nawigacji wewnętrznej opracowanego w ramach niniejszej pracy. Zaproponowany system, opierając się na nowoczesnych rozwiązaniach technologicznych, nie mógłby powstać bez wcześniejszych badań i osiągnięć innych autorów, które stanowiły punkt wyjścia do jego stworzenia. W szczególności, prace dotyczące metod nawigacji, sztucznej inteligencji oraz przetwarzania danych w przestrzeniach zamkniętych miały kluczowe znaczenie w kontekście rozwoju koncepcji, która znalazła swoje odzwierciedlenie w niniejszym projekcie.

Korzystanie z gotowych narzędzi, bibliotek programistycznych oraz algorytmów, które zostały opracowane w ramach innych badań, miało na celu przyspieszenie procesu implementacji, zapewniając jednocześnie solidne podstawy teoretyczne oraz sprawdzone metody działania. Dzięki tym zasobom możliwe było skoncentrowanie się na aspektach innowacyjnych, które stanowią wkład autora pracy, przy jednoczesnym zapewnieniu wysokiej jakości i efektywności zaprojektowanego systemu. W dalszej części rozdziału omówione zostaną konkretne technologie, modele oraz algorytmy, które były wykorzystywane podczas realizacji projektu, a także ich znaczenie w kontekście powstawania i rozwoju systemu nawigacji wewnętrznej.

4.2 Segmentacja obrazu

W proponowanym rozwiązaniu wykorzystano model sztucznej inteligencji, którego zadaniem jest segmentacja obrazu. Jest to jedno z najpopularniejszych zagadnień w dziedzinie widzenia komputerowego. Segmentacja obrazu jest kluczowym krokiem w przetwarzaniu planów budynków, gdzie celem jest rozróżnienie powierzchni umożliwiających poruszanie się od tych, które tego nie umożliwiają. Człowiek, patrząc na plan budynku, potrafi intuicyjnie rozpoznać takie obszary, jednak ręczne oznaczanie jest bardzo czasochłonne. Do wytrenowania i walidacji modelu użyto następujących popularnych zbiorów danych:

- MSD [21],
- PMD [22],
- MAS3K [23].

W literaturze wskazuje się, że modelem osiągającym najwyższą dokładność w segmentacji obrazu jest SAM2-UNet [24], co potwierdzają wyniki uzyskane przez twórców wymienionych zbiorów

danych [25]. Model ten działa wystarczająco skutecznie również w zadaniu segmentacji budynku na pomieszczenia, przewyższając takie rozwiązania jak SANet [26] oraz HETNet [27].

Wyniki badań, przedstawiające średni czas korekty błędów przez programistę w środowisku JIRA [28], zamieszczono w tabeli 4.1.

Tabela 4.1
Porównanie modeli segmentacji obrazu pod względem średniego czasu korekty błędów.

Model	Średni czas korekty [s]
SAM2-UNet	82.7
SANet	157.6
HETNet	132.1

4.3 Klasyfikacja gdzie wykonano zdjęcie

Jako kandydatów do wyboru odpowiedniego modelu przetestowano najlepsze modele na popularnych zbiorach danych treningowych, takich jak:

- ImageNet [29],
- ObjectNet [30],
- MNIST [31].

Przyjęto założenie, że model, który docelowo ma znaleźć się w systemie, nie może przekroczyć liczby 10 milionów parametrów. Najlepszym modelem według oficjalnych rankingów twórców powyższych zbiorów danych [32], spełniającym ten warunek, jest FixEfficientNet-B2 [33]. W zadaniu będącym podmiotem niniejszej pracy, gdzie liczba klas decyzyjnych jest zredukowana do dwóch, jego dokładność znacząco rośnie do 98.7%.

Dodatkowo obecność nieba na zdjęciu znacząco sugeruje modelowi, że zdjęcie zostało wykonane na zewnątrz, a niebo jest bardzo łatwo rozpoznać. Przetestowano również inne modele, takie jak MobileNet [34], wspomagany przez bibliotekę CvNets [35], oraz NoisyStudent (EfficientNet-B2) [36], jednak osiągały one nieznacznie gorsze dokładności.

Dokładne wartości dokładności wspomnianych modeli zostały przedstawione w tabeli 4.2.

Tabela 4.2
Porównanie modeli klasyfikujących pod kątem dokładności oraz liczby parametrów.

Model	Dokładność [%]	Liczba parametrów [mln]
FixEfficientNet-B2	98.7	9.2
MobileNet	97.8	5.5
NoisyStudent (EfficientNet-B2)	98.4	9.2

4.4 Detekcja obiektów charakterystycznych

Dla tego zadania również przyjęto założenie o zredukowanej wielkości proponowanych modeli do co najwyżej 10 milionów parametrów. Oczywiście wyborem jest tutaj próba sięgania po najlepsze modele specjalizujące się w wykrywaniu obiektów w czasie rzeczywistym [37].

Do wyboru potencjalnych kandydatów do testowania użyto popularnego zbioru danych MSCoco [38] i przetestowano 3 najlepsze modele na wcześniej wspomnianym zbiorze danych zdjęć z budynków

użyteczności publicznej. Jako potencjalnych kandydatów przetestowano następujące sieci neuronowe:

- DEIM-D-FINE-X+ [39],
- YOLO (w wersjach od 3 do 11) [40],
- D-FINE [41].

Wyniki dokładności wspomnianych modeli można zobaczyć w tabeli 4.3.

Tabela 4.3
Dokładność modeli w zadaniu wykrywania obiektów charakterystycznych na zbiorze danych zdjęć z budynków użyteczności publicznej.

Model	Dokładność (%)
DEIM-D-FINE-X+	58.3
YOLO (wersje 3–11)	57.9
D-FINE	58.1

4.5 Rozumienie tekstu na tablicach

Odczyt tekstu to klasyczne zagadnienie w dziedzinie sztucznej inteligencji, rozwiązywane za pomocą technologii OCR - Optical Character Recognition. Spośród popularnych zbiorów danych do uczenia modeli OCR wybrano następujące zestawy:

- COCO-text [42],
- IC19-Art [43],
- SVT [44].

Podobnie jak w poprzednio opisanych rozwiązaniach, model używany do analizy tekstu musi mieć mniej niż 10 milionów parametrów, aby mógł być wdrożony na urządzeniach mobilnych. Obecnie najlepszym kandydatem do tego zadania jest model Parseq [45], oparty na transformerze wizyjnym (ang. Vision Transformer) [46]. Do klasyfikacji tekstu zastosowano zmodyfikowany algorytm Levenshteina [47]

4.6 Algorytmy wyznaczania najlepszej ścieżki

Problem znajdowania najkrótszej ścieżki jest jednym z klasycznych zagadnień algorytmicznych. W celu rozwiązania tego zadania zastosowano dwa popularne algorytmy: algorytm Dijkstry [48] oraz algorytm A* [49].

4.7 Śledzenie przemieszczenia

W przytoczonych opracowaniach kluczową rolę w poprawie precyzji lokalizacji odgrywał Filtr Kalmana [50]. Zarówno w swojej pierwotnej, jak i rozszerzonej formie, filtr ten pozwala na analizę zaszumionych danych z wielu źródeł w celu uzyskania dokładnych wyników. Alternatywne podejście stanowi Filtr Madgwicka [51], który również jest szeroko stosowany w systemach lokalizacji i śledzenia.

Rozdział 5

Narzędzia programistyczne

5.1 Wprowadzenie

Korzystanie z gotowych narzędzi, bibliotek programistycznych oraz algorytmów, które zostały opracowane w ramach innych badań, miało na celu przyspieszenie procesu implementacji, zapewniając jednocześnie solidne podstawy teoretyczne oraz sprawdzone metody działania. Dzięki tym zasobom możliwe było skoncentrowanie się na aspektach innowacyjnych, które stanowią wkład autora pracy, przy jednoczesnym zapewnieniu wysokiej jakości i efektywności zaprojektowanego systemu. W dalszej części rozdziału omówione zostały konkretne technologie, modele oraz algorytmy, które były wykorzystywane podczas realizacji projektu, a także ich znaczenie w kontekście powstawania i rozwoju systemu nawigacji wewnętrznej.

5.2 C++

Jako język logiki aplikacji wybrano C++ w wersji 17 [52], gdyż jest on szybszy od większości innych popularnych języków programowania, takich jak Java [53], Python [54] czy Rust [55]. Ponadto C++ doskonale integruje się z narzędziem CMake [56], które odgrywa kluczową rolę w procesie budowania aplikacji wieloplatformowych.

5.3 CMake

CMake jako narzędzie do automatyzacji budowania aplikacji, umożliwia wygenerowanie plików konfiguracyjnych dostosowanych do różnych systemów operacyjnych i środowisk programistycznych [57]. Dzięki temu możliwe jest uniknięcie konieczności ręcznego pisania skomplikowanych skryptów budowania dla każdego systemu z osobna. W kontekście proponowanego systemu międzyplatformowego, CMake zapewnia:

1. **Obsługę różnych kompilatorów i środowisk:** CMake pozwala na wygenerowanie plików konfiguracyjnych dla szerokiej gamy kompilatorów, takich jak GCC, Clang czy MSVC, a także dla środowisk programistycznych, takich jak Visual Studio, Xcode czy Make. Dzięki temu kod źródłowy systemu może być kompilowany na wielu platformach bez konieczności modyfikacji.
2. **Łatwość konfiguracji zależności:** CMake umożliwia automatyczne pobieranie, budowanie i integrację zależności zewnętrznych, co znacznie upraszcza proces konfiguracji środowiska pracy.

3. **Modularność projektu:** CMake wspiera tworzenie projektów wielomodułowych, co ułatwia organizację kodu. Poszczególne moduły mogą być kompilowane i testowane niezależnie, co przyspiesza proces rozwoju oraz identyfikacji błędów.
4. **Integracja z testami jednostkowymi:** CMake wspiera integrację z popularnymi frameworkami do testów jednostkowych, takimi jak Google Test [58] czy Catch2 [59]. Dzięki temu możliwe jest automatyczne uruchamianie testów w trakcie budowania projektu, co pomaga w utrzymaniu wysokiej jakości kodu.
5. **Wsparcie dla cross-kompilacji:** Jedną z kluczowych cech CMake jest możliwość łatwej konfiguracji procesu cross-kompilacji, co jest szczególnie istotne w przypadku aplikacji przeznaczonych na różne platformy, takie jak Android, iOS czy systemy wbudowane. Dzięki temu programista może budować aplikację na system docelowy, korzystając z maszyny o zupełnie innej architekturze.
6. **Optymalizacja procesu budowania:** CMake obsługuje równoległe kompilowanie, co pozwala na znaczące skrócenie czasu budowania projektu. Narzędzie to umożliwia także precyzyjne definiowanie flag kompilatora, co pozwala na dostosowanie wydajności i rozmiaru aplikacji do specyficznych wymagań.

5.4 Djinni

Tworzenie interfejsu złożonego systemu informatycznego w C++ nie jest ogólnie przyjętą praktyką [60] ze względu na znacznie mniejszą liczbę bibliotek niż w językach dedykowanych do tego, takich jak Kotlin [61] czy Swift [62]. By zachować wydajność C++ oraz uzyskać dostęp do licznych bibliotek do tworzenia interfejsu w uprzednio wymienionych językach, należy wygenerować bridge code łączący interfejsy funkcji obu języków. Do tego zadania wykorzystano narzędzie Djinni [63], które oferuje szereg zalet w zakresie integracji międzyplatformowej:

1. **Automatyzacja procesu integracji:** Djinni pozwala na automatyczne generowanie kodu pośredniego, co znacznie przyspiesza proces integracji między językami i redukuje ryzyko błędów wynikających z ręcznego tworzenia mostów (bindings).
2. **Obsługa wielu platform:** Narzędzie wspiera popularne platformy, takie jak Android (Kotlin/Java), iOS (Swift/Objective-C) oraz systemy desktopowe. Dzięki temu możliwe jest tworzenie jednej logiki biznesowej w C++ i korzystanie z niej na różnych urządzeniach i systemach operacyjnych.
3. **Minimalizacja narzutu wydajnościowego:** Djinni generuje kod, który jest wysoce zoptymalizowany, co pozwala na minimalny narzut wydajnościowy podczas komunikacji pomiędzy językami. Dzięki temu aplikacje mogą działać z wysoką wydajnością.
4. **Przejrzystość i łatwość użycia:** Specyfikacja interfejsów w Djinni odbywa się za pomocą prostego pliku definiującego (IDL - Interface Definition Language), co czyni narzędzie łatwym w obsłudze, nawet dla początkujących użytkowników.
5. **Wsparcie dla szerokiego zakresu typów danych:** Djinni obsługuje wiele typów danych, w tym typy prymitywne (np. int, float), struktury, listy, mapy, a także typy opcjonalne i enumeracje. To pozwala na łatwe odwzorowanie złożonych struktur danych pomiędzy różnymi językami.

6. **Możliwość roszszerzania:** Djinni umożliwia dodawanie własnych rozszerzeń, co pozwala na dostosowanie generowanego kodu do specyficznych potrzeb projektu.

5.5 ONNX

Wybór C++ w wersji 17 jako języka logiki aplikacji wymaga doboru odpowiedniego formatu reprezentacji modeli sieci neuronowych. W kontekście niniejszej pracy przyjęto format Open Neural Network Exchange (ONNX) [64] jako podstawowy format przechowywania i wdrażania modeli.

Umożliwia on przechowywanie modeli wytrenowanych w popularnych frameworkach takich jak PyTorch [65], TensorFlow [66], scikit-learn [67] czy Keras [68]. Format ten wspiera ponad 150 operatorów matematycznych i funkcji, co pozwala na opisanie zarówno prostych, jak i złożonych sieci neuronowych. ONNX oferuje także szeroką kompatybilność z narzędziami optymalizacyjnymi, takimi jak ONNX Runtime [69], które umożliwia przyspieszenie wykonywania modeli na platformach z procesorami Intel, AMD, kartami graficznymi NVIDIA, a także na układach FPGA od firm takich jak Xilinx [70].

ONNX zapewnia interoperacyjność dzięki możliwości eksportu modeli z frameworków takich jak PyTorch (za pomocą funkcji `torch.onnx.export()`) oraz TensorFlow (za pomocą narzędzia TensorFlow-ONNX). Modele zapisane w formacie ONNX są w pełni kompatybilne z ONNX Runtime, co umożliwia ich wdrażanie zarówno w środowiskach serwerowych, jak i na urządzeniach mobilnych, takich jak Android i iOS.

Rozważając alternatywne formaty, istotne były ich wady i zalety w kontekście interoperacyjności, wsparcia sprzętowego i optymalizacji. Jednym z potencjalnych wyborów był format TensorFlow SavedModel. Umożliwia on jednocześnie zapisanie struktury modelu i wag, co ułatwia jego wdrożenie w systemach TensorFlow Serving [71]. Jednak format ten wymaga pełnego systemu TensorFlow do eksportu i uruchamiania modeli, co ogranicza elastyczność w heterogenicznych środowiskach. Dodatkowo wsparcie sprzętowe koncentruje się głównie na TPU i kartach graficznych obsługiwanych przez TensorRT.

Innym rozważanym rozwiązaniem był TorchScript, natywny dla PyTorch. TorchScript pozwala na eksportowanie modeli w formie skompilowanej, co zwiększa ich wydajność podczas wdrażania w środowiskach produkcyjnych. Jednak format ten jest ściśle związany z PyTorch, a narzędzia takie jak LibTorch [72] nie obsługują innych formatów, co ogranicza jego interoperacyjność.

Format Core ML [73], opracowany przez firmę Apple, został również przeanalizowany. Core ML cechuje się doskonałą integracją z ekosystemem Apple oraz wsparciem dla dedykowanego układu Neural Engine, co umożliwia wyjątkowo szybkie wykonywanie modeli na urządzeniach iOS i macOS. Jednak jego zastosowanie jest ograniczone wyłącznie do platform Apple, co wyklucza możliwość jego użycia w środowiskach wieloplatformowych.

Decyzja o wyborze formatu ONNX opiera się zatem na kluczowych faktach:

- **Interoperacyjność:** ONNX wspiera eksport modeli z frameworków takich jak TensorFlow, PyTorch, Keras i scikit-learn. Dzięki temu umożliwia migrację modeli między różnymi platformami bez konieczności zmiany kodu źródłowego.
- **Wsparcie sprzętowe:** ONNX Runtime obsługuje szeroką gamę platform, w tym procesory Intel (DNNL), NVIDIA GPU (CUDA), AMD, ARM oraz układy FPGA, co pozwala na optymalne wykorzystanie zasobów obliczeniowych w różnorodnych środowiskach.
- **Wsparcie operatorów:** ONNX umożliwia opisanie zaawansowanych architektur sieci neuronowych dzięki obsłudze ponad 150 operatorów matematycznych, co czyni go odpowiednim

zarówno dla prostych modeli klasyfikacyjnych, jak i złożonych sieci do przetwarzania obrazów czy języka naturalnego.

- **Wsparcie społeczności:** ONNX jest rozwijany przez firmy technologiczne, takie jak Microsoft, Facebook, Amazon i NVIDIA, co zapewnia stały rozwój oraz wdrażanie nowych funkcji.
- **Uniwersalność:** ONNX umożliwia wdrażanie modeli w środowiskach serwerowych, aplikacjach mobilnych oraz urządzeniach wbudowanych, co czyni go wszechstronnym rozwiązaniem dla projektów o szerokim zakresie zastosowań.

5.6 ONNX Runtime

ONNX Runtime jest środowiskiem uruchomieniowym stworzonym przez Microsoft w celu wykonywania modeli zapisanych w formacie ONNX. Zaprojektowano je z myślą o optymalizacji wydajności oraz wsparciu dla szerokiego zakresu platform sprzętowych i środowisk wdrożeniowych, co czyni je kluczowym komponentem w projektach wymagających szybkiego przetwarzania modeli uczenia maszynowego.

ONNX Runtime obsługuje różnorodne backendy obliczeniowe [74], w tym:

- **Procesory CPU:** Dzięki integracji z bibliotekami optymalizacyjnymi, takimi jak Intel DNNL (oneDNN) oraz OpenBLAS, ONNX Runtime zapewnia wysoką wydajność na procesorach wielordzeniowych.
- **Karty graficzne GPU:** Wsparcie dla NVIDIA CUDA i TensorRT pozwala na znaczące przyspieszenie inferencji modeli, zwłaszcza w przypadku złożonych sieci neuronowych przetwarzających duże ilości danych, takich jak obrazy czy sekwencje tekstowe.
- **Układy FPGA i inne platformy:** ONNX Runtime obsługuje także układy FPGA oraz inne niestandardowe akceleratory, co umożliwia jego wykorzystanie w systemach wbudowanych.

Jednym z kluczowych elementów ONNX Runtime jest wsparcie dla *Graph Optimizations* [75]. Proces ten obejmuje transformacje grafów obliczeniowych modeli, takie jak składanie operatorów (ang. *operator fusion*), eliminacja redundantnych obliczeń oraz statyczne przypisywanie wartości.

ONNX Runtime wspiera również wykonywanie modeli w trybie równoległym oraz w systemach rozproszonych. Opcje takie jak *intra-op parallelism* i *inter-op parallelism* pozwalają na jednoczesne wykonywanie wielu operacji, co zwiększa przepustowość obliczeń. W aplikacjach wdrażanych na serwerach chmurowych ONNX Runtime wykorzystuje skalowalność chmury do obsługi dużych wolumenów danych.

Porównując ONNX Runtime do alternatywnych środowisk uruchomieniowych, takich jak TensorFlow Serving czy PyTorch Serve, można zauważyć jego większą uniwersalność. TensorFlow Serving, choć wysoce zoptymalizowany dla modeli TensorFlow, nie oferuje wsparcia dla innych formatów bez dodatkowych konwersji. Podobnie PyTorch Serve jest ściśle związany z frameworkiem PyTorch. W przeciwieństwie do nich, ONNX Runtime może obsługiwać modele z różnych frameworków bez potrzeby konwersji do formatu natywnego.

Dodatkową zaletą ONNX Runtime jest możliwość integracji z narzędziami do przyspieszania inferencji, takimi jak *ONNX Runtime Web* [76], które umożliwiają wykonywanie modeli w przeglądarkach internetowych, oraz *ONNX Runtime Mobile* [77], które zostało zoptymalizowane pod

kątem urządzeń mobilnych. Dzięki temu ONNX Runtime wspiera wdrażanie modeli w szerokim zakresie środowisk – od serwerów chmurowych po urządzenia końcowe.

Wybór ONNX Runtime w kontekście tej pracy wynika zatem z jego wydajności, wsparcia dla zaawansowanych optymalizacji oraz szerokiej kompatybilności z różnymi platformami sprzętowymi.

Rozdział 6

Międzyplatformowość

6.1 Wprowadzenie

Jednym z podstawowych założeń proponowanego systemu jest zapewnienie międzyplatformowości, co oznacza możliwość poprawnego działania systemu niezależnie od systemu operacyjnego urządzenia, architektury procesora czy innych specyficznych właściwości sprzętowych. Takie podejście pozwala na stworzenie uniwersalnego rozwiązania, które może być z powodzeniem wykorzystywane zarówno na urządzeniach mobilnych z systemem Android i iOS, jak i na innych platformach, takich jak systemy wbudowane czy komputery osobiste.

Aby osiągnąć ten cel, w projekcie uniknięto korzystania z wewnętrznych, specyficznych dla danego systemu operacyjnego funkcji, które mogłyby ograniczać jego uniwersalność. Zamiast tego zastosowano standardowe biblioteki i technologie, które są szeroko dostępne i kompatybilne z wieloma platformami. Dzięki temu proponowane rozwiązanie cechuje się wysoką elastycznością i może zostać wdrożone w bardzo krótkim czasie do dowolnego istniejącego systemu zewnętrznego, niezależnie od jego specyfiki.

Zastosowane podejście przynosi szereg korzyści, w tym zmniejszenie kosztów i czasu potrzebnego na dostosowanie systemu do nowych środowisk, a także łatwość utrzymania i aktualizacji oprogramowania. Rozwiązanie to jest szczególnie istotne w kontekście aplikacji mobilnych i systemów nawigacyjnych, gdzie różnorodność sprzętu i oprogramowania użytkowników wymaga zastosowania wszechstronnych i adaptowalnych technologii.

6.2 Moduł `IndoorNavigationService`

Rozwiązaniem opisanych wyżej problemów jest zaimplementowany w ramach niniejszej pracy moduł `IndoorNavigationService`. Opracowany system umożliwia lokalizację użytkownika w przestrzeni oraz śledzenie jego przemieszczenia w czasie rzeczywistym, wykorzystując dane z czujników urządzenia (akcelerometru, żyroskopu i magnetometru) oraz obraz z kamery.

Zastosowanie tego podejścia pozwala na eliminację konieczności instalowania dodatkowej infrastruktury nawigacyjnej w budynkach, co znacząco obniża koszty wdrożenia systemu oraz zwiększa jego skalowalność. Ponadto, dzięki wykorzystaniu wbudowanych sensorów i algorytmów przetwarzania obrazu, system może działać w różnorodnych warunkach, niezależnie od dostępności sygnału GPS czy innych zewnętrznych źródeł danych lokalizacyjnych.

Poniżej znajduje się szczegółowy opis funkcji dostępnych w module `IndoorNavigationService`. Każda funkcja została przedstawiona z uwzględnieniem parametrów wejściowych, oczekiwanych wyników oraz analizy złożoności obliczeniowej, która ma kluczowe znaczenie dla oceny wydajności systemu na różnych platformach.

1. `create()`

Tworzy nową instancję serwisu `IndoorNavigationService`.

Parametry: Brak.

Złożoność obliczeniowa: $O(1)$, ponieważ funkcja jedynie inicjalizuje obiekt i alokuje podstawowe zasoby wymagane do działania modułu.

Zastosowanie: Funkcja służy jako punkt początkowy dla dalszego korzystania z modułu nawigacyjnego, zapewniając poprawną inicjalizację zasobów systemowych.

2. `loadShops(DirectoryPath, prefix)`

Wczytuje informacje o sklepach z podanego katalogu, uwzględniając prefiks w nazwach plików.

Parametry:

- `DirectoryPath (string)`: Ścieżka do katalogu zawierającego dane o sklepach.
- `prefix (string)`: Prefiks określający, które pliki powinny być wczytane.

Złożoność obliczeniowa: $O(n)$, gdzie n to liczba punktów charakterystycznych w katalogu spełniających kryteria prefiksu.

3. `loadPoints(DirectoryPath, prefix)`

Wczytuje punkty nawigacyjne z podanego katalogu na podstawie prefiksu.

Parametry:

- `DirectoryPath (string)`: Ścieżka do katalogu zawierającego dane o punktach nawigacyjnych.
- `prefix (string)`: Prefiks określający pliki do wczytania.

Złożoność obliczeniowa: $O(n)$, gdzie n to liczba punktów nawigacyjnych (pixeli) w plikach o rozszerzeniu `.png`.

Uwagi: Funkcja zapewnia walidację danych wejściowych, co zwiększa niezawodność systemu kosztem nieznacznego wzrostu czasu przetwarzania.

4. `loadModels(detectorPath, ocrPath)`

Ładuje modele sztucznej inteligencji używane do detekcji i rozpoznawania tekstu.

Parametry:

- `detectorPath (string)`: Ścieżka do modelu detekcyjnego.
- `ocrPath (string)`: Ścieżka do modelu OCR.

Złożoność obliczeniowa: $O(1)$ dla inicjalizacji ścieżek, natomiast czas wczytywania modeli zależy od ich rozmiaru (m), co daje $O(m)$.

Uwagi: Modele są ładowane do pamięci RAM, co minimalizuje czas dostępu podczas ich wykorzystania w klasyfikacji obiektów.

5. `classifyDetections(timestamp, fieldOfView, img_width, img_height, compass, boxes)`

Klasyfikuje wykryte obiekty na podstawie ich cech.

Parametry:

- `timestamp (double)`: Czas wykonania detekcji.
- `fieldOfView (double)`: Pole widzenia kamery.

- `img_width, img_height (int)`: Rozdzielczość obrazu.
- `compass (double)`: Kąt wskazania kompasu.
- `boxes (vector<Box>)`: Wykryte obszary na obrazie.

Złożoność obliczeniowa: $O(b)$, gdzie b to liczba wykrytych obiektów w obrazie.

Uwagi: Funkcja stosuje algorytmy klasyfikacyjne z użyciem modeli Sztucznej inteligencji, co może być czasochłonne przy dużej liczbie obiektów.

6. `cameraPosition(PreviousPoint, timestamp, fieldOfView, img_width, img_height, compass, boxes)`

Określa pozycję kamery na podstawie wcześniejszych punktów i wykrytych obiektów.

Parametry:

- `PreviousPoint (Point)`: Poprzednia pozycja.
- `timestamp (double)`: Czas wykonania operacji.
- `fieldOfView (double)`: Kąt widzenia kamery.
- `img_width, img_height (int)`: Rozdzielczość obrazu.
- `compass (double)`: Kąt wskazania kompasu.
- `boxes (vector<Box>)`: Wykryte obiekty.

Złożoność obliczeniowa: $O(b)$, gdzie b to liczba wykrytych obiektów.

Uwagi: Dokładność pozycji zależy od jakości danych wejściowych, takich jak stabilność obrazu i precyzja kompasu.

7. `getPath(start, destination, compassAngle)`

Funkcja generuje optymalną ścieżkę od punktu początkowego `start` do punktu końcowego `destination`, uwzględniając kąt kompasu `compassAngle`.

Parametry:

- `start (Point)`: Punkt początkowy ścieżki.
- `destination (Point)`: Punkt końcowy ścieżki.
- `compassAngle (float)`: Kąt kompasu w stopniach.

Złożoność obliczeniowa: Algorytm wykorzystuje A^* do wyznaczenia najkrótszej ścieżki w grafie 3D:

$$O(n \log n + e),$$

gdzie n to liczba węzłów, a e liczba krawędzi.

Uwagi: Złożoność zależy od rozmiaru siatki (`grid size`). Najczęściej jest to liczba pixeli w pliku.

8. `getAllShops()`

Funkcja zwraca listę wszystkich zarejestrowanych sklepów w systemie. Lista zawiera szczegóły każdego sklepu, takie jak jego położenie oraz inne metadane.

Parametry: Brak. Funkcja działa na danych załadowanych do pamięci. **Złożoność obliczeniowa:** $O(n)$, gdzie n to liczba sklepów.

Uwagi: Funkcja przeszukuje i zwraca pełną listę obiektów.

9. `updateLocation(point, step, smallThreshold, mediumThreshold, largeThreshold, azimuth, heightDiff)`

Funkcja aktualizuje lokalizację użytkownika w oparciu o liczbę kroków, kąty oraz różnicę wysokości.

Parametry:

- `point (Point)`: Aktualna lokalizacja użytkownika.
- `step (float)`: Wielkość kroku w metrach.
- `smallThreshold, mediumThreshold, largeThreshold (float)`: Progi różnic wysokości.
- `azimuth (float)`: Aktualny azymut w stopniach.
- `heightDiff (float)`: Różnica wysokości.

Złożoność obliczeniowa: $O(1)$

10. `walking(point, timestamp, accX, accY, accZ, gyroX, gyroY, gyroZ, pitch, roll, azimuth, heightDiff, stepCount)`

Funkcja oblicza nową pozycję użytkownika na podstawie danych z akcelerometru, żyroskopu oraz liczby kroków.

Parametry:

- `point (Point)`: Aktualna pozycja użytkownika.
- `timestamp (long)`: Czas pomiaru.
- `accX, accY, accZ (float)`: Wartości przyspieszenia w osiach X, Y, Z.
- `gyroX, gyroY, gyroZ (float)`: Wartości żyroskopu w osiach X, Y, Z.
- `pitch, roll, azimuth (float)`: Kąty orientacji urządzenia.
- `heightDiff (float)`: Różnica wysokości.
- `stepCount (int)`: Liczba kroków użytkownika.

Złożoność obliczeniowa: $O(n)$, gdzie n to liczba pamiętanych ostatnich punktów na mapie, gdzie znajdował się użytkownik

11. `setParameters(usingBarometer, usingStepCounter, usingCompassInFindMe)`

Funkcja umożliwia ustawienie parametrów systemowych, takich jak wykorzystanie barometru, krokomierza i kompasu w funkcji `findMe`.

Parametry:

- `usingBarometer (bool)`: Flaga określająca, czy wykorzystywać barometr.
- `usingStepCounter (bool)`: Flaga określająca, czy wykorzystywać krokomierz.
- `usingCompassInFindMe (bool)`: Flaga określająca, czy kompas jest używany w funkcji lokalizacji.

Złożoność obliczeniowa: $O(1)$

Rozdział 7

Początkowa lokalizacja i pozycjonowanie

7.1 Wprowadzenie

Rozpoznanie początkowej lokalizacji użytkownika stanowi kluczowy element działania systemu pozycjonowania w budynkach. Zdefiniowanie dokładnego punktu startowego jest niezbędne do zapewnienia prawidłowego funkcjonowania aplikacji. Bez tego system nie jest w stanie wyznaczyć optymalnej ścieżki prowadzącej użytkownika do celu, co znacząco ogranicza jego funkcjonalność i praktyczne zastosowanie. Problem ten nabiera szczególnego znaczenia w złożonych środowiskach wewnętrznych, takich jak galerie handlowe, centra konferencyjne czy szpitale, gdzie brak dostępu do standardowych systemów GPS uniemożliwia korzystanie z klasycznych metod lokalizacji.

Początkowa lokalizacja użytkownika wymaga zastosowania zaawansowanych technik integrujących dane z wielu źródeł, takich jak mapy budynków, analiza obrazu oraz odczyty z sensorów urządzenia mobilnego. Konieczne jest również zrozumienie kontekstu przestrzennego i umiejscowienie użytkownika w trójwymiarowym układzie współrzędnych, co wymaga odwzorowania pozycji dwuwymiarowej na pełne współrzędne 3D.

Podstawą skutecznego pozycjonowania w budynkach jest dostęp do spójnego i jednolitego formatu map przestrzennych. Każda mapa musi być precyzyjnie zdefiniowana, uwzględniając strukturę budynku, w tym poszczególne piętra, przejścia, punkty orientacyjne, takie jak sklepy, wejścia i windy, oraz granice obszarów dostępnych i niedostępnych. Mapy te muszą również umożliwiać jednoznaczne odwzorowanie współrzędnych 2D obiektów wykrytych przez kamerę na współrzędne 3D. Dodatkowo konieczna jest poprawna reprezentacja różnicy odległości między obiektami oraz wysokości między piętrami oraz zapewnić zgodność z rzeczywistym układem budynku.

7.2 Mapowanie budynków

Większość budynków użyteczności publicznej, takich jak muzea czy galerie handlowe, udostępnia publicznie swoje plany na stronach internetowych. Ma to na celu ułatwienie wizytującym wcześniejszego zaplanowania wizyty lub zapoznania się z ofertą. Dodatkowo każdy budynek został zbudowany na podstawie precyzyjnych planów architektonicznych. Proponowany w niniejszej pracy system wykorzystuje oba rozwiązania, w zależności od dostępności planów architektonicznych. W przypadku ich publicznej dostępności zaleca się ich wykorzystanie, natomiast w przeciwnym wypadku należy skorzystać z planów udostępnianych na stronach internetowych budynku.

W niniejszym przykładzie wykorzystano plany Galerii Pestka w Poznaniu [78]. Plany każdego piętra zostały zapisane w postaci zrzutów ekranu i umieszczone w pamięci wewnętrznej urządzenia. Format i rozszerzenie zapisanych obrazów mogą być dowolne, jednak ze względu na ich popularność



Rysunek 7.1
Oryginalny plan budynku. [78]

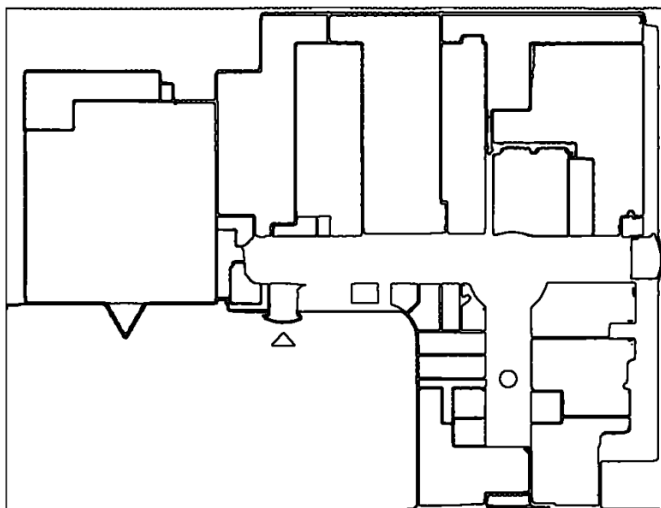
sugeruje się użycie formatów .png lub .jpg. Przed dalszym wykorzystaniem plany te muszą zostać odpowiednio przetworzone.

7.3 Rozpoznawanie obiektów na planie budynku

W ramach zadania lokalizacji wewnętrznej przyjęto, że budynek może składać się z następujących elementów, którym przypisano odpowiednie identyfikatory numeryczne:

1. Podłoga (powierzchnia przeznaczona do poruszania się).
2. Przestrzeń dostępna wyłącznie dla osób upoważnionych (np. serwerownia, pokój ochrony).
3. Ściany nieprzezroczyste.
4. Ściany przezroczyste (rozdzielenie jest istotne, gdyż przez niektóre ściany można dostrzec obiekty znajdujące się po drugiej stronie).
5. Przestrzeń otwarta (np. otwory w stropach umożliwiające widok na inne piętra, stosowane głównie w celach estetycznych).
6. Windy.
7. Schody ruchome.
8. Obiekty charakterystyczne (np. rzeźby, obrazy, witryny sklepowe).
9. Toalety jako szczególny przypadek punktów charakterystycznych.

Podstawowym krokiem jest rozróżnienie powierzchni umożliwiających poruszanie się od tych, które tego nie umożliwiają. Człowiek, patrząc na plan budynku, potrafi intuicyjnie rozpoznać takie obszary, jednak ręczne oznaczanie jest bardzo czasochłonne. Dlatego w proponowanym rozwiązaniu wykorzystano wcześniej wspomniany model sztucznej inteligencji SAM2-UNet [24], którego zadaniem jest segmentacja obrazu.



Rysunek 7.2
Segmentacja obrazu na planie budynku.

7.4 Klasyfikacja obiektów

Po wygenerowaniu listy wielokątów wykrytych na obrazie przez model segmentacji obrazu, konieczne jest ich prawidłowe sklasyfikowanie. Przyjęcie prostego założenia, że każdy wykryty duży wielokąt (powyżej 5% powierzchni obrazka) jest osobnym pomieszczeniem, okazuje się wystarczające w tym przypadku. Wynika to bezpośrednio z przyjętych założeń oraz ograniczonego zbioru elementów, które mogą zostać sklasyfikowane. W danych wejściowych znajduje się poprawna mapa budynku oraz nałożona siatka segmentacji obrazu, co oznacza, że każdy wykryty przez model, a następnie poprawiony przez człowieka wielokąt faktycznie należy do jednego z dziewięciu typów obiektów zdefiniowanych w podrozdziale Mapowanie budynków. Duży wielokąt musi zatem stanowić przestrzeń przeznaczoną do chodzenia, ponieważ pozostałe elementy, takie jak obrazy, rzeźby czy toalety, są znacznie mniejsze i zawsze znajdują się wewnątrz innych pomieszczeń.

Klasyfikacja drzwi do pomieszczeń opiera się na zmodyfikowanym algorytmie DBSCAN [79] do grupowania obiektów w klastry. Wektor cech w tym kontekście odnosi się do wartości liczbowych uzyskanych z kanałów RGBA na oryginalnym obrazie. Przyjęto również założenie, że w każdym standardowym pomieszczeniu powierzchnia ścian jest większa niż powierzchnia drzwi i okien. Z tego powodu klaster z większą liczbą punktów uznaje się za ściany, a klaster mniejszy za wejścia do pomieszczenia.

Aby ułatwić użytkownikowi zrozumienie systemu, środek każdego wejścia do pomieszczenia oznaczono jako punkt charakterystyczny. Stanowi on potencjalny cel nawigacyjny, ponieważ człowiek, chcąc dotrzeć do danego pomieszczenia, najprawdopodobniej chce wejść do niego właśnie przez drzwi.

Następnie przyjęto założenie, że każda duża otwarta przestrzeń to korytarz, czyli główna przestrzeń przeznaczona do chodzenia. Pozostałe elementy, takie jak toalety, schody ruchome, windy, szklone sklepy czy inne punkty charakterystyczne, często nie są oznaczone na planach architektonicznych budynku. Oznaczenie ich pozostawiono ekspertowi, który, kierując się swoją wiedzą dziedzinową, może w krótkim czasie oznaczyć wszystkie wymagane punkty.

Podobnie, określenie przezroczystości ścian również powierzono ekspertowi. Może on wskazać, czy każda ściana znajdująca się obok korytarza powinna zostać oznaczona jako przezroczysta. Jest to szczególnie pożądane w przestrzeniach handlowych, gdzie sklepy często eksponują swoją ofertę



Rysunek 7.3
Budynek z oznaczonymi pomieszczeniami i miejscami specjalnymi.

przez przeszklone ściany.

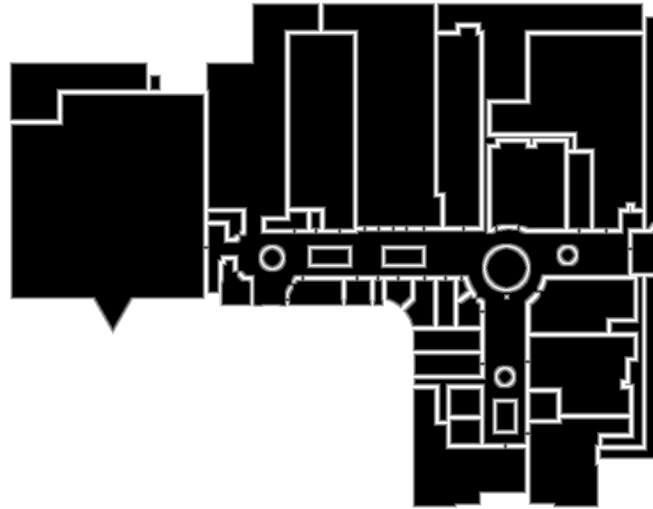
Jako ostatnią czynność ekspertowi pozostawiono oznaczenie pomieszczeń i korytarzy niedostępnych dla zwykłego użytkownika. Miejsca te nie będą udostępniane użytkownikom bez odpowiednich uprawnień, ale mogą być dostępne dla personelu technicznego lub administracyjnego. Ekspert oznacza wszystkie miejsca zgodnie z przyjętą konwencją kolorów:

- Białe: #ffffff — Obszary poza mapą
- Żółty: #fff200 — Miejsca przeznaczone do chodzenia
- Pomarańczowy: #ff7f27 — Dziury, przez które nie można przejść
- Niebieski: #00a2e8 — Środek drzwi
- Czarny: #000000 — Ściany
- Zielony: #22b14c — Szklane szyby
- Czerwony: #ed1c24 — Przestrzeń dla osób upoważnionych
- Brązowy: #b97a57 — Schody ruchome
- Fioletowy: #A349A4 — Windy

7.5 Punkty preferowane

Każdy punkt oznaczony na żółto jest miejscem, na którym użytkownik może stawać. Większość odbiorców końcowych systemu wolałaby jednak poruszać się środkiem korytarza niż ścieżką bardzo blisko ściany. Z drugiej strony w ciasnych pomieszczeniach, takich jak alejki sklepowe, nie da się uniknąć wąskich korytarzy, dlatego nie należy zakazywać użytkownikom poruszania się po nich.

Proponowany w niniejszej pracy system wyposażono w algorytm, który prowadzi użytkownika możliwie najbardziej komfortową drogą. Szczegółowy opis tego mechanizmu znajduje się w sekcji 8.2.1.



Rysunek 7.4

Graficzne przedstawienie wartości funkcji kary dla omawianego planu

Podczas mapowania budynku wsparciem dla tych algorytmów jest system punktów preferowanych. Każdemu z dozwolonych miejsc przypisywana jest kara za znajdowanie się w jego położeniu.

Oznaczenia używane we wzorach:

- δ – odległość punktu od najbliższej ściany, wyrażona w jednostkach formatu mapy, najczęściej pixelach,
- κ – współczynnik skali mapy, nadawany przez eksperta, bezwymiarowy,
- $\phi(\delta)$ – funkcja kary za znajdowanie się w punkcie o odległości δ od najbliższej ściany.

W przypadku podstawowego systemu, gdzie nie uwzględnia się skali mapy, funkcja kary ma postać:

$$\phi(\delta) = 1 - \frac{\max(5, \delta)}{5}. \quad (7.1)$$

Po uwzględnieniu skali mapy κ , wzór kary przyjmuje następującą postać:

$$\phi(\delta, \kappa) = 1 - \frac{\max(5, \frac{\delta}{\kappa})}{5}. \quad (7.2)$$

7.6 Klasyfikacja, gdzie zostało wykonane zdjęcie

Początkowym zadaniem jest stwierdzenie, czy użytkownik w ogóle znajduje się w budynku. Może się okazać, że pewne charakterystyczne punkty, takie jak obrazy, reklamy, banery, znajdują się zarówno w budynku, jak i poza nim. Jeśli użytkownik próbowałby się zlokalizować po tych charakterystycznych punktach, skanując je z zewnątrz, cały system doszedłby do błędnych wniosków dotyczących początkowego położenia i lokalizacji. Jak wykazano wcześniej, w takiej sytuacji również nie można polegać na sygnale GPS, dlatego proponowanym podejściem w niniejszej pracy jest użycie modelu sztucznej inteligencji klasyfikującego obrazy na dwie klasy: wewnątrz i na zewnątrz.

Jest to znane zadanie w środowisku badaczy sztucznej inteligencji. Natomiast w tym konkretnym zadaniu trzeba sprawić, aby wybrany model był w stanie działać na urządzeniu mobilnym. Dlatego musi on mieć niewielką liczbę parametrów i dodatkowo krótki czas inferencji arbitralnie przyjęty na co najwyżej 2 sekundy. Ponadto w tym konkretnym przypadku istnieją tylko dwie docelowe klasy decyzyjne.

Do zadaniu będącego podmiotem niniejszej pracy, wykorzystano uprzednio wspomniany model FixEfficientNet-B2 [33]. Liczba klas decyzyjnych jest zredukowana do tylko dwóch, dlatego jego dokładność znacząco również się poprawia i rośnie do 98.7%. Dodatkowo obecność nieba na zdjęciu znacząco sugeruje modelowi, że zdjęcie zostało wykonane na zewnątrz, a niebo jest bardzo łatwo rozpoznać.

Dla praktycznych zastosowań 98.7% to wystarczająco dużo, zwłaszcza gdy w przypadku błędnej klasyfikacji użytkownik może wywołać funkcję lokalizującą jeszcze raz, zmniejszając prawdopodobieństwo pomyłki. Przyjmując poniższe oznaczenia:

- P_b – prawdopodobieństwo błędnej klasyfikacji,
- p – dokładność modelu, wyrażona jako ułamek dziesiętny (tutaj $p = 0.987$),
- n – liczba prób wykonanych przez użytkownika.

Prawdopodobieństwo błędnej klasyfikacji wyrażone jest wzorem

$$P_b = (1 - p)^n. \quad (7.3)$$

i jak można łatwo zauważyć, maleje wykładniczo wraz ze wzrostem liczby prób n , co zapewnia minimalną szansę na pomyłkę modelu.

7.7 Detekcja obiektów charakterystycznych

Jeśli ustalono, że użytkownik znajduje się wewnątrz budynku, należy rozpoznać jego otoczenie poprzez wykrycie obiektów charakterystycznych w jego najbliższym otoczeniu. W kontekście lokalizacji wewnętrznej, wykrywanie obiektów takich jak obrazy, tabliczki informacyjne, czy elementy wyposażenia wewnątrz staje się kluczowym zadaniem w rozwiązywaniu problemu pozycjonowania użytkownika w przestrzeni. Jednym z głównych wyzwań jest zróżnicowanie wielkości i kształtu obiektów, które mogą być rozpoznawane na zdjęciach, a także zmienne warunki zewnętrzne, które wpływają na ich detekcję.

Dokładność najlepszego wcześniej wspomnianego modelu do tego zadania, DEIM-D-FINE-X+ [39], wynosi około 60%. Choć nie wydaje się to być zbyt dużo, warto zwrócić uwagę, że taka dokładność jest wystarczająca do realizacji zadanego problemu w praktyce. Nie jest konieczne dokładne wykrycie wszystkich obiektów charakterystycznych w obrębie danego pomieszczenia, gdyż do poprawnej lokalizacji użytkownika wystarczy rozpoznać tylko kilka kluczowych elementów. Często już wykrycie dwóch obiektów, a w niektórych przypadkach nawet tylko jednego, jest wystarczające, aby precyzyjnie oszacować pozycję osoby wewnątrz budynku.

Algorytm lokalizacji oparty na wykrywaniu obiektów charakterystycznych zostanie opisany w sekcji 7.8, gdzie szczegółowo omówione zostanie, jak wykryte elementy wpływają na algorytm ustalania pozycji. Warto jednak zauważyć, że wiele obiektów wykrywanych w kontekście tego zadania, takich jak obrazy, tabliczki czy rośliny doniczkowe, charakteryzuje się niewielkimi rozmiarami w porównaniu do innych elementów otoczenia. Z tego powodu, ich detekcja bywa bardziej wymagająca, co może negatywnie wpływać na dokładność modelu. Takie obiekty mogą być łatwo pominięte w wyniku szumów w obrazach lub zmienności warunków oświetleniowych, które mogą sprawić, że staną się mniej widoczne lub nieczytelne.

W związku z tym, wprowadzenie wag dla wykrytych obiektów, oznaczających ich znaczenie w algorytmie lokalizacyjnym, mogłoby pozwolić na bardziej precyzyjne oszacowanie rzeczywistej wartości dokładności detekcji. Obiekty o wyższej wadze mogłyby mieć większy wpływ na wynik

**Rysunek 7.5**

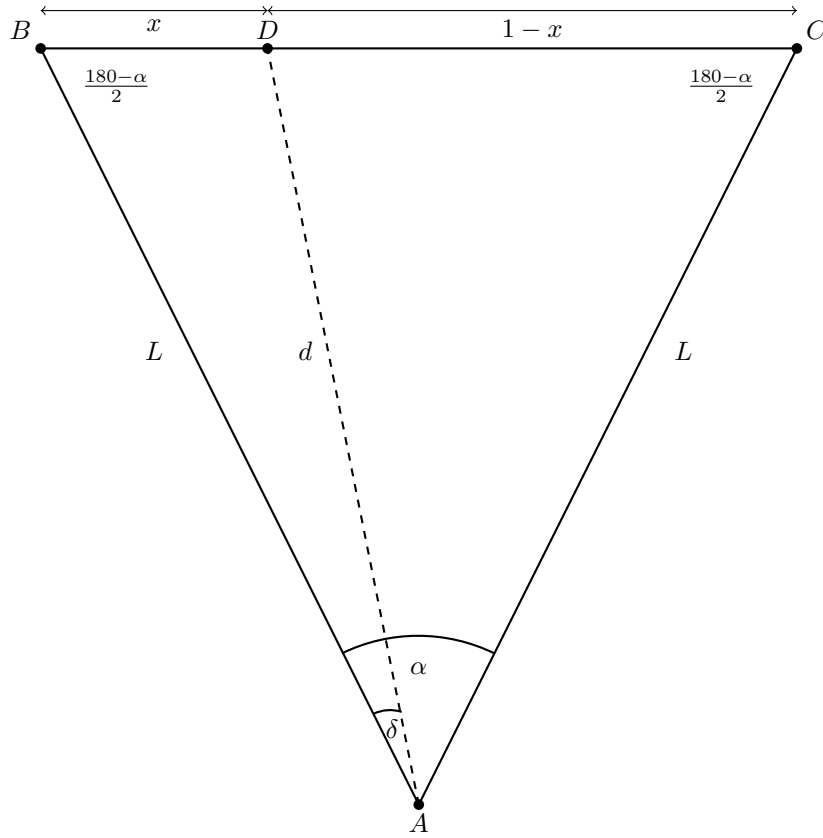
Wykryte obiekty w przestrzeni handlowej

końcowy, podczas gdy obiekty mniej istotne mogłyby być traktowane przez obliczaną miarę dokładności z mniejszą uwagą.

Niemniej jednak, w tym zadaniu zastosowano standardową miarę dokładności powszechnie używaną w statystyce oraz uczeniu maszynowym. Chociaż metody te są szeroko stosowane i łatwe do implementacji, warto zauważyć, że wprowadzenie wag dla wykrytych obiektów może znacznie poprawić prognozowaną rzeczywistość wykrywania, szczególnie w bardziej złożonych i dynamicznych środowiskach.

7.8 Lokalizacja w oparciu o pozycję obiektów charakterystycznych

Gdy znane jest pole widzenia (ang. *field of view*) kamery w danym urządzeniu, na którym osadzony jest system, widoczne są co najmniej dwa punkty charakterystyczne, a także znany jest kąt patrzenia kamery względem północy θ , można jednoznacznie określić pozycję użytkownika na mapie. Aby to zrobić, należy najpierw ustalić kąt względem północy, pod którym użytkownik patrzy na każdy z wykrytych obiektów. Dla jednego obiektu sytuacja przedstawiona jest na rysunku 7.6, gdzie α to pole widzenia (*field of view*), a punkt A to pozycja użytkownika. Odcinki AB oraz AC odpowiadają maksymalnym kątom patrzenia kolejno w lewo i w prawo, dla zadanego pola widzenia. Odcinek BC jest projekcją obrazu rzeczywistego w 3D na siatkę 2D kamery. Dla ułatwienia obliczeń, przeskalowano cały trójkąt tak, aby odcinek BC miał długość 1. Punkt D to zauważony obiekt, a zatem jego odległość od lewego krańca zdjęcia wynosi x , a od prawego $1 - x$. Na rysunku zaznaczone zostały również inne wartości przydatne do obliczeń, takie jak kąty $\angle ABC$ oraz $\angle ACB$, a także długości odcinków AB , AD i AC . Należy zatem znaleźć kąt δ , który jest kątem w przestrzeni dwuwymiarowej, oznaczającym kąt, pod którym użytkownik patrzy na dany wykryty obiekt.



RYSUNEK 7.6: Pozycja użytkownika względem wykrytego obiektu

Na początku wyprowadza się wzór na długość odcinka L . Rozważa się trójkąt o wierzchołkach A , B , C oraz D . Odcinki AB i AC mają tę samą długość.

Obliczenie miary kąta δ przeprowadza się w sposób analogiczny do podejścia przedstawionego w pracy *Cosine law at the atomic scale: Toward realistic simulations of Knudsen diffusion* [80]. W szczególności wykorzystuje się twierdzenie cosinusów, które dla trójkąta o bokach L , L oraz BC wyraża się równaniem:

$$BC^2 = L^2 + L^2 - 2 \cdot L \cdot L \cdot \cos(\alpha) \quad (7.4)$$

gdzie α to kąt między odcinkami AB i AC . Po przekształceniu równania:

$$BC^2 = 2L^2(1 - \cos(\alpha)) \quad (7.5)$$

Ponieważ odcinek BC jest skalowany do jednostkowej długości (tj. $BC = 1$), otrzymuje się:

$$1 = 2L^2(1 - \cos(\alpha)) \quad (7.6)$$

Po dalszym przekształceniu:

$$L^2 = \frac{1}{2(1 - \cos(\alpha))} \quad (7.7)$$

Następnie stosuje się tożsamość trygonometryczną:

$$1 - \cos(\alpha) = 2 \sin^2\left(\frac{\alpha}{2}\right) \quad (7.8)$$

Po podstawieniu tej tożsamości do poprzedniego równania uzyskuje się:

$$L^2 = \frac{1}{4 \sin^2\left(\frac{\alpha}{2}\right)} \quad (7.9)$$

Zatem:

$$L = \frac{1}{2 \sin\left(\frac{\alpha}{2}\right)} \quad (7.10)$$

Następnie oblicza się długość odległości d między punktami B i D . Korzysta się ze wzoru opisanego przez twierdzenie cosinusów, które dla trójkąta o bokach L , d oraz x jest opisane następującym równaniem:

$$d = \sqrt{L^2 + x^2 - 2 \cdot L \cdot x \cdot \cos(\alpha)} \quad (7.11)$$

Aby obliczyć kąt δ między punktem użytkownika a wykrytym obiektem, stosuje się ponownie twierdzenie cosinusów. Z definicji kosinusa kąta δ dla trójkąta o bokach L , d i x , otrzymuje się:

$$\cos(\delta) = \frac{L^2 + d^2 - x^2}{2 \cdot L \cdot d} \quad (7.12)$$

Z powyższego wzoru oblicza się kąt δ :

$$\delta = \cos^{-1}\left(\frac{L^2 + d^2 - x^2}{2 \cdot L \cdot d}\right) \quad (7.13)$$

Na koniec wyznacza się rzeczywisty kąt względem kierunku północnego, oznaczany jako γ , który określa położenie wykrytego obiektu. Korzysta się tutaj z faktu, że kąt δ reprezentuje odchylenie od lewej granicy pola widzenia do obiektu, natomiast wyrażenie $\theta - \frac{\alpha}{2}$ określa kąt między północą a lewą granicą pola widzenia.

W związku z tym rzeczywisty kąt względem północy γ obliczany jest według następującego wzoru:

$$\gamma = \theta - \frac{\alpha}{2} + \delta \quad (7.14)$$

W analizie rozważane są dwa obiekty o znanych współrzędnych (x_1, y_1) oraz (x_2, y_2) . Dla każdego z tych obiektów wyznaczane są kąty γ_1 i γ_2 , które określają rzeczywiste położenie obiektów względem użytkownika oraz północy.

Obliczenia tych kątów przeprowadza się analogicznie do podejścia przedstawionego w pracy *Straight Lines and Angles that Move, Students' Ideas that Touch and Add* [81]. A zatem równanie prostej przechodzącej przez punkt (x, y) pod kątem γ względem osi poziomej można wyrazić w postaci ogólnej:

$$A \cdot x + B \cdot y + C = 0, \quad (7.15)$$

gdzie współczynniki A , B oraz C obliczane są jako:

$$A = \sin(\gamma), \quad B = -\cos(\gamma), \quad C = -(x \cdot A + y \cdot B). \quad (7.16)$$

Dla dwóch obiektów, wyznaczane są współczynniki prostych:

$$A_1 = \sin(\gamma_1), \quad B_1 = -\cos(\gamma_1), \quad C_1 = -(x_1 \cdot A_1 + y_1 \cdot B_1), \quad (7.17)$$

$$A_2 = \sin(\gamma_2), \quad B_2 = -\cos(\gamma_2), \quad C_2 = -(x_2 \cdot A_2 + y_2 \cdot B_2). \quad (7.18)$$

Punkt przecięcia tych prostych, jeśli istnieje, obliczany jest przy pomocy wzoru na wyznacznik macierzy współczynników:

$$\det = A_1 \cdot B_2 - A_2 \cdot B_1. \quad (7.19)$$

Jeśli $\det = 0$, oznacza to, że proste są równoległe lub pokrywają się, co wyklucza istnienie punktu przecięcia. W przeciwnym przypadku współrzędne punktu przecięcia (x, y) obliczane są ze wzorów:

$$x = \frac{B_2 \cdot (-C_1) - B_1 \cdot (-C_2)}{\det}, \quad (7.20)$$

$$y = \frac{A_1 \cdot (-C_2) - A_2 \cdot (-C_1)}{\det}. \quad (7.21)$$

Po wyznaczeniu punktu przecięcia sprawdzana jest poprawność tego punktu względem kątów γ_1 i γ_2 . Kąty γ_1 oraz γ_2 są obliczane na podstawie współrzędnych obiektów i punktu przecięcia (x, y) jako:

$$\gamma'_1 = \arctan\left(\frac{y_1 - y}{x_1 - x}\right), \quad \gamma'_2 = \arctan\left(\frac{y_2 - y}{x_2 - x}\right). \quad (7.22)$$

Następnie porównywane są kąty γ'_1 i γ'_2 z wartościami γ_1 i γ_2 . Weryfikacja odbywa się z uwzględnieniem tolerancji numerycznej $\varepsilon = 10^{-6}$, wynikającej z ograniczeń obliczeń na liczbach zmiennoprzecinkowych. Sprawdzane jest, czy:

$$|\gamma'_1 - \gamma_1| < \varepsilon \quad \text{oraz} \quad |\gamma'_2 - \gamma_2| < \varepsilon. \quad (7.23)$$

Jeśli oba warunki są spełnione, punkt przecięcia jest uznawany za poprawne położenie użytkownika. Tolerancja ε pozwala uwzględnić niewielkie różnice wynikające z zaokrągleń w obliczeniach.

7.8.1 Analiza punktów charakterystycznych wykrytych na różnych piętrach

W przypadku, gdy wykryte punkty charakterystyczne znajdują się na różnych piętrach, nie wpływa to na opisany wcześniej algorytm obliczania współrzędnych x i y pozycji użytkownika. Natomiast obliczenie współrzędnej z wymaga analizy rozkładu wykrytych punktów na poszczególnych piętrach.

Proponowane podejście polega na obliczeniu entropii wartości z dla wszystkich wykrytych punktów. Jeśli entropia przekracza przyjęty próg 0.75, oznacza to, że wykryte punkty znajdujące się na innych piętrach mogły zostać błędnie zidentyfikowane lub wynikają z widoku przez otwory w stropie pomiędzy piętrami. W takim przypadku jako współrzędną z przyjmuje się wartość najczęściej występującą wśród wykrytych punktów.

Z kolei, gdy entropia jest mniejsza od wartości progowej 0.75, użytkownik jest proszony o ponowne rozejrzenie się kamerą w celu wykrycia większej liczby punktów charakterystycznych, co umożliwi dokładniejsze określenie pozycji.

7.9 Rozumienie tekstu na tablicach

Kiedy detektor obiektów charakterystycznych identyfikuje część obrazu jako szyld, reklamę lub tablicę informacyjną, konieczne jest zrozumienie zawartego na niej tekstu w celu poprawnego przyporządkowania obiektu do jego miejsca na mapie. W zbiorze uczącym, opisanym wcześniej, wszystkie takie obiekty zawierały maksymalnie trzy wyrazy stanowiące logiczną całość, takie jak „Punkt Informacyjny”, „Toaleta”, „Peron 2” czy „Czas na Herbatę”. W związku z tym proces ten nie wymaga głębokiego rozumienia semantyki tekstu, lecz jedynie klasyfikacji na podstawie znanych kategorii oraz porównania z tekstami zapisanymi w przygotowanej mapie.

Odczyt tekstu to klasyczne zagadnienie w dziedzinie sztucznej inteligencji, rozwiązywane za pomocą technologii OCR - Optical Character Recognition, którą implementuje wcześniej wspomniany model Parseq [45]

Po odczytaniu tekstu konieczna jest jego klasyfikacja. Jednym z częstych błędów występujących w procesie OCR jest powtarzanie ostatniego znaku, co może prowadzić do nieprawidłowej interpretacji wyników. Na przykład, zamiast prostego „radio”, model może rozpoznać tekst jako „radioccccccccccc”. Usuwanie powtarzających się znaków na końcu tekstu nie jest poprawnym rozwiązaniem, ponieważ istnieją przypadki, takie jak nazwa sklepu „CCC” czy logotyp T-Mobile zawierający elementy „T...”. W celu rozwiązania tego problemu zastosowano zmodyfikowany algorytm Levenshteina [47], który uwzględnia takie przypadki. Algorytm ten można opisać za pomocą następujących kroków:

7.9.1 Algorytm Levenshteina z uwzględnieniem błędów końcowych

1. **Inicjalizacja macierzy odległości:** Tworzona jest macierz dwuwymiarowa D , w której element $D(i, j)$ przechowuje minimalny koszt edycji przekształcenia pierwszych i znaków wzorca S w pierwsze j znaków wykrytego tekstu T . Pierwszy wiersz i pierwsza kolumna macierzy są inicjalizowane wartościami odpowiadającymi kosztowi usunięcia lub dodania znaków:

$$D(0, j) = j, \quad D(i, 0) = i, \quad \forall i, j \geq 0.$$

2. **Porównanie znaków:** Dla każdej pary znaków z wzorca S oraz wykrytego tekstu T , obliczana jest wartość $D(i, j)$:

- Jeśli znaki są zgodne ($S[i] = T[j]$), wartość w komórce $D(i, j)$ zostaje skopiowana z $D(i - 1, j - 1)$.
- Jeśli znaki są różne ($S[i] \neq T[j]$), wartość $D(i, j)$ jest obliczana jako minimum spośród:

$$D(i - 1, j) + 1, \quad D(i, j - 1) + 1, \quad D(i - 1, j - 1) + 1,$$

gdzie kolejno reprezentują one koszty usunięcia, wstawienia i zamiany znaku.

3. **Korygowanie błędów końcowych:** Podczas przeszukiwania wstecznego analizowane są znaki na końcu wykrytego tekstu T . Jeśli ostatnie znaki $T[j]$ nie mają odpowiednika w S , ich koszt jest traktowany jako pojedynczy błąd, niezależnie od liczby takich znaków. Dzięki temu minimalizowane są błędy wynikające z powtarzania końcowych znaków.
4. **Obliczanie końcowej odległości:** Zwracana jest skorygowana odległość edycyjna d oraz współczynnik podobieństwa σ , obliczany jako:

$$\sigma = 1 - \frac{d}{|S| + 1},$$

gdzie $|S|$ to długość wzorca S .

Powyższy algorytm skutecznie radzi sobie z błędami wynikającymi z nadmiarowych znaków na końcu wykrytego tekstu, jednocześnie zachowując wysoką dokładność w identyfikacji tekstów z mapy.

7.10 Dane treningowe

Jak wspomniano wcześniej, metody lokalizowania się w budynku opierają się w dużej części na modelach sztucznej inteligencji. Jakość takich modeli jest bezpośrednio związana z jakością, różnorodnością oraz reprezentatywnością zbioru treningowego. Dane treningowe dla używanych modeli powinny być jak najbardziej zbliżone do tych, które pojawiają się przy rzeczywistym używaniu systemu. W przeciwnym razie modele mogą działać poprawnie w kontrolowanych warunkach testowych, ale nie poradzą sobie w środowisku rzeczywistym.

Z tego powodu, dla uzyskania jak najlepszej jakości, wykonano 1700 zdjęć przy użyciu trzech smartfonów, dbając o różnorodność systemów operacyjnych oraz warunków. Do tego zadania użyto następujących aparatów:

- Xiaomi POCO F3,
- iPhone 16 Pro Max,
- Samsung Galaxy S20.

Zdjęcia wykonano w trzech różnych rozdzielczościach, a liczba zdjęć z każdego telefonu została podsumowana w poniższej tabeli:

TABELA 7.1: Liczba zdjęć w zależności od telefonu i rozdzielczości

Telefon	1920x1080	2560x1440	3840x2160
Xiaomi POCO F3	200	150	100
iPhone 16 Pro Max	250	200	150
Samsung Galaxy S20	300	200	150
Razem	750	550	400

Zdjęcia zostały wykonane w różnych warunkach świetlnych, o różnych porach dnia, kątach nachylenia oraz pozycji użytkownika (na parkingu, poza budynkiem, w środku, w sklepie itp.). Taka różnorodność pozwala modelowi nauczyć się rozpoznawać obrazy w sytuacjach odbiegających od tych idealnych, co jest kluczowe dla zwiększenia jego odporności na zakłócenia.

Ważnym aspektem podczas tworzenia zbioru treningowego była również kontrola nad balansem klas i cech reprezentowanych w danych. Oznacza to, że zdjęcia musiały obejmować zarówno różnorodne konteksty (np. korytarze, sklepy, wyjścia ewakuacyjne), jak i różne warunki techniczne (np. różne rozdzielczości zdjęć, kąty nachylenia kamery). Taka konstrukcja danych treningowych zmniejsza ryzyko tzw. przetrenowania, które występuje, gdy model zbyt ściśle dopasowuje się do specyficznych wzorców w danych treningowych, kosztem ogólności.

Należy również zaznaczyć, że liczba danych treningowych oraz ich zróżnicowanie są fundamentalne dla poprawnego działania algorytmu w praktycznych zastosowaniach. Przy niewystarczającej ilości danych lub ich niskiej jakości model może stać się podatny na błędy w rzeczywistym środowisku, co prowadzi do obniżenia jego przydatności. Z kolei dobrze skonstruowany zbiór treningowy pozwala nie tylko osiągnąć wysoką dokładność modelu, ale także zwiększyć jego zdolność do generalizacji i radzenia sobie z niespodziewanymi scenariuszami.



Rysunek 7.7
Przykładowe dane treningowe przedstawiające różne piętra budynku

Rozdział 8

Śledzenie przemieszczenia

8.1 Wprowadzenie

Gdy znana jest pozycja użytkownika i trasa do wyznaczonego przez niego punktu docelowego, należy śledzić jego ruch w przestrzeni. Najprostszym rozwiązaniem byłaby ciągła lokalizacja po punktach charakterystycznych, jednak jednym z wymagań funkcjonalnych systemu jest umożliwienie użytkownikowi trzymanie telefonu poziomo. W wielu budynkach użyteczności publicznej zabrania się nagrywania i robienia zdjęć w środku. Choć proponowany w niniejszej pracy system nie zapisuje żadnych danych z kamery dłużej niż na czas potrzebny do zlokalizowania użytkownika, a także nie wysyła ich na żaden zewnętrzny system, zachowanie odbiorców, którzy będą trzymać telefon w orientacji pionowej, może budzić podejrzenia służb ochrony oraz służb mundurowych, w zależności od budynku. Dlatego należy skorzystać z innych danych pozwalających na śledzenie przemieszczenia.

Jak wspomniano wcześniej, zadaniem proponowanego systemu jest nieużywanie zewnętrznych źródeł sygnału, za to dozwolone jest korzystanie z czujników wbudowanych w smartfony, takich jak akcelerometr, żyroskop, kompas i barometr. Nieprzetworzone dane z tych czujników są niewystarczające do konstrukcji poprawnie działającego systemu AHRS (ang. Attitude and Heading Reference System). [82] Kolejnym czujnikiem wbudowanym w smartfony jest licznik kroków, który działa z opóźnieniem w systemie iOS [83] oraz Android [84].

8.2 Algorytmy wyznaczania najlepszej ścieżki

Problem znajdowania najkrótszej ścieżki jest jednym z klasycznych zagadnień algorytmicznych. W celu rozwiązania tego zadania zastosowano dwa popularne algorytmy: algorytm Dijkstry [48] oraz algorytmu A* [49].

8.2.1 Trasa na jednej kondygnacji

W nawigacji wewnętrznej istotnym zagadnieniem jest unikanie niepożądanych obszarów, takich jak miejsca blisko ścian. Choć punkty te są technicznie dostępne dla użytkownika, znacząco zmniejszają komfort poruszania się. W celu minimalizacji tego efektu wprowadzono system kar, który uwzględnia odległość euklidesową oraz dodatkowy współczynnik kary oparty na wcześniej zdefiniowanej macierzy kar. Odległość euklidesowa d między dwoma punktami (x_1, y_1) i (x_2, y_2) jest obliczana jako:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}. \quad (8.1)$$

Wprowadzona kara p zależy od wartości w macierzy kar w punkcie (x, y) i jest określona wzorem:

$$p = 1 + \frac{p(x, y)}{255}, \quad (8.2)$$

gdzie $p(x, y)$ to wartość kary w punkcie (x, y) .

Ostateczna wartość odległości d_f z uwzględnieniem kary jest obliczana w zależności od kierunku ruchu:

1. Dla ruchu w kierunku poziomym lub pionowym (góra, dół, lewo, prawo):

$$d_f = d + p. \quad (8.3)$$

2. Dla ruchu po przekątnej (skosy):

$$d_f = d + \sqrt{2} \cdot p. \quad (8.4)$$

Do obliczania optymalnych ścieżek na pojedynczej kondygnacji wykorzystano algorytm A^* , który uwzględnia zarówno odległość d , jak i opisany system kar.

8.2.2 Trasa po wielu kondygnacjach

Problem wyznaczania trasy pomiędzy punktami znajdującymi się na różnych kondygnacjach jest bardziej złożony. Standardowe podejście oparte wyłącznie na odległości euklidesowej d nie jest wystarczające, ponieważ ignoruje ograniczenia związane z koniecznością przechodzenia między piętrami wyłącznie za pomocą schodów lub wind.

W celu rozwiązania tego problemu wprowadzono następujące podejście:

1. **Estymacja odległości do punktów charakterystycznych.** Dla każdego punktu na kondygnacji obliczane są odległości d_c do najbliższych punktów charakterystycznych, takich jak schody czy windy. Pozwala to na określenie, które punkty są kluczowe dla zmiany piętra.
2. **Budowa grafu punktów charakterystycznych.** Punkty charakterystyczne każdej kondygnacji są wierzchołkami grafu, a krawędzie e między nimi odpowiadają najkrótszym trasom wyznaczonym za pomocą algorytmu Dijkstry. Długości krawędzi e_{ij} uwzględniają zarówno odległość d , jak i potencjalne kary p wynikające z komfortu użytkownika.
3. **Wyznaczanie optymalnej trasy.** Jeśli punkt docelowy znajduje się na innej kondygnacji, wyznaczana jest lista punktów charakterystycznych na optymalnej trasie prowadzącej do celu. Dla każdego odcinka między punktami na tej samej kondygnacji ponownie stosowany jest algorytm A^* , aby uwzględnić lokalne warunki. Ostateczna odległość d_f uwzględnia zarówno lokalne kary, jak i odległość do punktów charakterystycznych.
4. **Przejście między kondygnacjami.** Jeśli dwa kolejne punkty charakterystyczne na trasie stanowią początek i koniec schodów lub windy, ich połączenie jest traktowane jako bezpośrednie przejście.

Dzięki takiemu podejściu algorytm uwzględnia zarówno lokalne warunki na danej kondygnacji, jak i globalną strukturę budynku. Pozwala to na efektywne wyznaczanie tras, które są intuicyjne i wygodne dla użytkownika, nawet w skomplikowanych budynkach o wielu piętrach.

8.3 Parametry algorytmu detekcji kroków

Przed rozpoczęciem opisu algorytmu detekcji kroków przedstawiono kluczowe parametry i symbole używane w równaniach:

- g – przyspieszenie ziemskie, ustalone na podstawie średnich odczytów z akcelerometru w okresie stabilności.
- $a(t)$ – aktualna wartość przyspieszenia z akcelerometru w chwili t .
- $\gamma(t)$ – aktualna wartość średniej magnitudy sygnału z żyroskopu w chwili t .
- $c(t)$ – aktualny kąt telefonu względem północy w chwili t .
- A_H – próg wyznaczający wysokie przyspieszenie dla kroku, $1.2g$.
- A_L – próg wyznaczający niskie przyspieszenie dla kroku, $0.9g$.
- T_{min} – minimalny czas między kolejnymi krokami, 0.3 sekundy.
- t_L – czas ostatniego wystąpienia niskiego przyspieszenia.
- t_S – czas ostatniego wykrytego kroku.
- θ_S – kąt między osią Y a kierunkiem skoku przyspieszenia powyżej $1.2g$.
- d – kierunek kroku jako suma kąta z kompasu i θ_S .

8.4 Algorytm detekcji kroków

W poniższych wzorach stosuje się normalizację odczytów analogicznie do podejścia zastosowanego w pracy *Advancing space-based gravitational wave astronomy: Rapid parameter estimation via normalizing flows* [85], gdzie techniki normalizacji zostały użyte do poprawy dokładności estymacji parametrów w detekcji fal grawitacyjnych.

1. Ustalenie wartości przyspieszenia ziemskiego g :

- a) Odczytaj dane z akcelerometru (a_x, a_y, a_z) .
- b) Oblicz zmiany przyspieszenia:

$$\Delta a_x = |a_x - a_{x,prev}|, \quad \Delta a_y = |a_y - a_{y,prev}|, \quad \Delta a_z = |a_z - a_{z,prev}|. \quad (8.5)$$

- c) Jeżeli $\Delta a_x, \Delta a_y, \Delta a_z < 0.5$, zaktualizuj wartość średnią:

$$g = \sqrt{\frac{\sum_{i=1}^N a_{x,i}^2 + a_{y,i}^2 + a_{z,i}^2}{N}}. \quad (8.6)$$

2. Normalizacja odczytów akcelerometru:

$$a'_x = \frac{a_x}{g}, \quad a'_y = \frac{a_y}{g}, \quad a'_z = \frac{a_z}{g}. \quad (8.7)$$

3. Oblicz magnitudy przyspieszenia i żyroskopu:

$$|a'(t)| = \sqrt{a'^2_x + a'^2_y + a'^2_z}, \quad (8.8)$$

$$|\gamma(t)| = \sqrt{\gamma_x^2 + \gamma_y^2 + \gamma_z^2}. \quad (8.9)$$

4. Detekcja kroku:

- a) Jeżeli $|a'(t)| < A_L$, zapamiętaj czas t_L .
- b) Jeżeli $|a'(t)| > A_H$ oraz:

$$|a'(t)| > 1.5 \cdot |\gamma(t)|, \quad (8.10)$$

$$t - t_S > T_{min}, \quad (8.11)$$

$$t_L > t_S, \quad (8.12)$$

zarejestruj krok i zaktualizuj t_S .

5. Detekcja kierunku kroku:

- a) Odczytaj aktualny kąt telefonu względem północy $c(t)$.
- b) Wykryj kierunek przyspieszenia dla kroku:

$$\theta_S = \arctan\left(\frac{a_y}{a_x}\right), \quad (8.13)$$

gdzie $|a'(t)| > A_H$.

- c) Oblicz kierunek kroku:

$$d = c(t) + \theta_S. \quad (8.14)$$

Kroki od 2 do 5 są powtarzane w pętli dla każdego nowego zestawu danych z czujników.

8.5 Analiza liczby przeszkód pomiędzy 2 punktami

Wspomniane wcześniej czujniki, a w szczególności kompas, zwany również magnetometrem, popełniają błędy ze względu na podatność na zakłócenia [86]. W efekcie, w pewnych sytuacjach, takich jak próba określenia kierunku kroku prowadzącego przez ścianę, wskazanie kierunku może być niemożliwe. Aby zapewnić poprawność trajektorii kroku, stosuje się algorytm weryfikujący, który oblicza liczbę obiektów niemożliwych do sforsowania pomiędzy dwoma zadanymi punktami. Algorytm ten działa analogicznie do podejścia zastosowanego w pracy *Modeling the Line: Bresenham's Algorithm, 1962–87* [87], gdzie do analizy pikseli na siatce rastrowej wykorzystano metodę dyskretnego wyznaczania linii prostej. Podobnie, w proponowanym rozwiązaniu sprawdzane są kolejne komórki przestrzeni pomiędzy punktami, co pozwala na efektywną detekcję przeszkód i weryfikację poprawności trajektorii ruchu.

Algorytm działa w dwóch przypadkach:

1. Punkty startowy i końcowy znajdują się na różnych piętrach ($z_1 \neq z_2$).
2. Punkty startowy i końcowy znajdują się na tym samym piętrze ($z_1 = z_2$).

8.5.1 Różne piętra ($z_1 \neq z_2$)

W przypadku, gdy punkty startowy i końcowy znajdują się na różnych piętrach:

1. Oblicz różnice współrzędnych:

$$\Delta x = x_2 - x_1, \quad \Delta y = y_2 - y_1, \quad \Delta z = z_2 - z_1. \quad (8.15)$$

2. Znajdź przedział pięter:

$$z_{\min} = \min(z_1, z_2), \quad z_{\max} = \max(z_1, z_2). \quad (8.16)$$

Dla każdej całkowitej wartości z w przedziale $[z_{\min}, z_{\max}]$, oblicz punkt przecięcia linii łączącej (x_1, y_1, z_1) i (x_2, y_2, z_2) z płaszczyzną piętra:

$$t = \frac{z - z_1}{\Delta z}, \quad x = x_1 + t \cdot \Delta x, \quad y = y_1 + t \cdot \Delta y. \quad (8.17)$$

3. Zaokrąglaj współrzędne x i y do najbliższych wartości całkowitych:

$$x_{\text{round}} = \text{round}(x), \quad y_{\text{round}} = \text{round}(y). \quad (8.18)$$

4. Sprawdź, czy punkt $(x_{\text{round}}, y_{\text{round}}, z)$ znajduje się na przeszkodzie (np. ścianie lub obiekcie, którego nie można sforsować). Jeśli tak, oznacz przejście jako zablokowane.

Powyższa procedura, w przypadku dwóch różnych pięter, weryfikuje legalność przemieszczenia na dystansie dłuższym niż pojedynczy krok. Ma to szczególne znaczenie w sytuacji, gdy użytkownik znajduje się na schodach ruchomych lub w windzie i pozostaje nieruchomy. W takim przypadku następny krok zostanie wykonany dopiero na kolejnym piętrze. Procedura ta zapewnia, że użytkownik rzeczywiście zmienił piętro w wyznaczonym do tego miejscu.

8.5.2 To samo piętro ($z_1 = z_2$)

Jeśli punkty startowy i końcowy znajdują się na tym samym piętrze, algorytm weryfikuje przejście wzdłuż linii łączącej (x_1, y_1) i (x_2, y_2) . Parametry linii określa równanie ogólne:

$$Ax + By + C = 0, \quad (8.19)$$

gdzie:

$$A = y_2 - y_1, \quad B = x_1 - x_2, \quad C = x_2 \cdot y_1 - x_1 \cdot y_2. \quad (8.20)$$

Algorytm działa następująco:

1. Dla każdej wartości x_i z zakresu:

$$x_i = \min(x_1, x_2) + i, \quad i \in \{0, 1, \dots, |x_2 - x_1|\}, \quad (8.21)$$

oblicz odpowiadającą wartość y_i :

$$y_i = -\frac{A \cdot x_i + C}{B}. \quad (8.22)$$

Zaokrąglaj y_i do najbliższej wartości całkowitej $\text{round}(y_i)$ i sprawdź, czy punkt $(x_i, \text{round}(y_i), z_1)$ znajduje się na przeszkodzie.

2. Analogicznie, dla każdej wartości y_j z zakresu:

$$y_j = \min(y_1, y_2) + j, \quad j \in \{0, 1, \dots, |y_2 - y_1|\}, \quad (8.23)$$

oblicz odpowiadającą wartość x_j :

$$x_j = -\frac{B \cdot y_j + C}{A}. \quad (8.24)$$

Zaokrąglaj x_j do najbliższej wartości całkowitej $\text{round}(x_j)$ i sprawdź, czy punkt $(\text{round}(x_j), y_j, z_1)$ znajduje się na przeszkodzie.

W powyższych krokach, zakres iteracji i i j zapewnia przejście przez wszystkie wartości współrzędnych x i y w odpowiednich zakresach $[x_1, x_2]$ oraz $[y_1, y_2]$. Wynikowe punkty są sprawdzane pod kątem obecności przeszkód na danym piętrze z_1 .

Jeśli którakolwiek z powyższych weryfikacji wykryje przeszkodę pomiędzy punktami, przejście zostaje uznane za niemożliwe. W przeciwnym razie trajektoria jest poprawna.

8.6 Wykrywanie zmiany piętra niezależnie od krokomierza

Dodatkowe informacje o zmianie piętra można uzyskać z barometru wbudowanego w urządzenie mobilne [88], który mierzy wysokość nad poziomem morza. Barometr umożliwia wykrycie nagłych zmian wysokości, co wskazuje na przemieszczanie się użytkownika windą lub schodami ruchomymi. Na początku działania aplikacji system odczytuje wysokość nad poziomem morza i przypisuje ją aktualnemu piętru użytkownika. Początkowo zakłada się, że użytkownik nie korzysta z windy ani schodów ruchomych (`onElevatorOrEscalator = false`). Wysokości pięter są zapisane w definicji budynku, co pozwala na obliczenie wysokości pozostałych pięter.

Parametry używane w algorytmie:

- δ_h – różnica wysokości między aktualną wartością odczytaną z barometru a wartością przewidywaną dla piętra.
- t – aktualny znacznik czasu.
- t_c – czas ostatniej kalibracji wysokości.
- t_s – czas ostatniej zmiany piętra.
- $t_{\Delta c} = 300\text{ s}$ – minimalny odstęp czasu między kalibracjami.
- $t_{\Delta s} = 5\text{ s}$ – margines czasowy pozwalający na opuszczenie windy lub schodów ruchomych.
- k – liczba kroków zarejestrowanych przez krokomierz.
- k_0 – liczba kroków w momencie wejścia na windę lub schody ruchome.
- h_p – lista wysokości zarejestrowanych podczas ostatnich kroków.
- $d = 0.6\text{ m}$ – maksymalna różnica wysokości uznawana za brak zmiany piętra.
- Δ_h – różnica wysokości między kolejnymi piętrami w budynku
- f_z – obecne piętro, na którym znajduje się użytkownik.
- f_i – najbliższe piętro, wyznaczone na podstawie różnicy wysokości.
- e – status wskazujący, czy użytkownik korzysta z windy lub schodów ruchomych.

Algorytm detekcji zmiany piętra:

1. Wykrywanie wejścia na windę lub schody ruchome:

a) Oblicz różnicę wysokości względem wszystkich pięter:

- Wyznacz różnicę wysokości $\delta_h^{(i)} = |\delta_h - h_i|$ dla każdego piętra i .
- Jeśli użytkownik przebywa na obecnym piętrze f_z , zastosuj dodatkową wagę do różnicy by zapobiec przedwczesnej zmianie piętra:

$$\delta_h^{(f_z)} = \delta_h^{(f_z)} - 0.5. \quad (8.25)$$

b) Znajdź najbliższe piętro:

$$f_i = \arg \min_i \delta_h^{(i)}. \quad (8.26)$$

c) Jeśli użytkownik zmienił piętro ($f_z \neq f_i$):

- Oznacz $e = 1$, co oznacza wejście na windę lub schody ruchome.
- Ustaw $t_c = t$ oraz $t_s = t$.
- Zarejestruj liczbę kroków w momencie wejścia ($k_0 = k$).

2. Wykrywanie zejścia z windy lub schodów ruchomych:

a) Jeśli użytkownik korzysta z windy lub schodów ruchomych ($e = 1$), sprawdź, czy wysokość wykryta przez czujnik przestała się zmieniać oraz czy wykonano co najmniej 5 kroków dla stabilizacji:

$$|\delta_h - h_p[0]| < d \quad \text{oraz} \quad k - k_0 \geq 5. \quad (8.27)$$

b) Jeśli warunki są spełnione:

- Oznacz $e = 0$, co oznacza opuszczenie windy lub schodów ruchomych.
- Zaktualizuj Δ_h na podstawie δ_h .
- Ustaw $t_c = t$.

3. Korekta dryftu czujnika:

a) Jeśli czas od ostatniej kalibracji przekracza $t_{\Delta c}$, sprawdź:

- Jeśli użytkownik był na windzie lub schodach ruchomych dłużej niż $t_{\Delta s}$, przeprowadź kalibrację:
 - Zaktualizuj Δ_h na podstawie δ_h .
 - Ustaw $t_c = t$ oraz $t_s = t$.
 - Oznacz $e = 0$.
- Jeśli użytkownik opuścił windę lub schody ruchome w czasie krótszym niż $t_{\Delta s}$, kontynuuj monitorowanie.

b) Jeśli użytkownik nie korzysta z windy ani schodów ruchomych:

- Przeprowadź kalibrację:
 - Zaktualizuj Δ_h na podstawie δ_h .
 - Ustaw $t_c = t$.

8.7 Korekcja kumulujących się błędów

Po wykryciu przemieszczenia użytkownika konieczne jest odpowiednie zaktualizowanie jego pozycji na mapie. Może się jednak zdarzyć, że zarejestrowane przemieszczenie nie jest możliwe do wykonania z bieżącej pozycji użytkownika, ponieważ ta jest obciążona błędem. Znane są jednak wcześniejsze przemieszczenia użytkownika oraz mapa, na której się porusza. Ponieważ użytkownik rzeczywiście wykonał krok, oznacza to, że było to fizycznie możliwe. Dlatego należy znaleźć najbliższą możliwą do wykonania pozycję użytkownika, z której możliwe jest wykonanie wykrytego przemieszczenia.

Do rozwiązania tego problemu zastosowano algorytm Iterative Closest Point (ICP), [89], pierwotnie służący do znajdowania wektora translacji i rotacji między dwoma zbiorami punktów w

przestrzeni 3D, minimalizując błąd różnicy odległości między parami punktów. Algorytm ten był wielokrotnie modyfikowany od czasu jego pierwotnego opracowania, [90], i znajduje szerokie zastosowanie w dopasowywaniu chmur punktów.

W niniejszej pracy zaproponowano wykorzystanie algorytmu ICP do dopasowywania zbiorów odcinków w przestrzeni 2D. Redukcja wymiarowości, a co za tym idzie liczby stopni swobody, zmniejsza liczbę lokalnych minimów i punktów siodłowych funkcji kosztu, co sprawia, że algorytm gradientowy, jakim jest ICP, rzadziej wpada w minima lokalne i częściej znajduje rozwiązanie globalne. Kluczowa różnica w proponowanym podejściu polega na modyfikacji funkcji znajdowania najbliższych możliwych do przejścia odcinków, opisanej poniżej.

8.7.1 Mechanizm wykrywania najbliższych sąsiadów

Funkcja znajdowania najbliższych sąsiadów (*findNearestNeighbors*) została dostosowana do potrzeb zadania korekcji błędów przemieszczenia. Jej działanie opisano w sposób formalny poniżej.

Dla każdego kroku zadanego jako odcinek $L = (\mathbf{p}_1, \mathbf{p}_2)$, gdzie $\mathbf{p}_1 = (x_1, y_1)$ i $\mathbf{p}_2 = (x_2, y_2)$ są punktami końcowymi, stosowany jest następujący algorytm:

1. **Sprawdzanie legalności punktów:** Dla każdego punktu końcowego $\mathbf{p}_i \in \{\mathbf{p}_1, \mathbf{p}_2\}$ sprawdzana jest jego legalność na podstawie funkcji:

$$\mathcal{L}(\mathbf{p}_i) = \begin{cases} 1, & \text{jeśli } \mathbf{p}_i \text{ jest legalny,} \\ 0, & \text{w przeciwnym razie.} \end{cases} \quad (8.28)$$

2. **Korekcja nielegalnych punktów:** Jeśli $\mathcal{L}(\mathbf{p}_i) = 0$, wyszukiwany jest najbliższy punkt legalny $\mathbf{q}_i = \arg \min_{\mathbf{q} \in \mathcal{M}} \|\mathbf{p}_i - \mathbf{q}\|$, gdzie $\mathcal{M}$ oznacza zbiór punktów legalnych na mapie.

Punkt $\mathbf{p}_i$ zostaje zaktualizowany do wartości $\mathbf{q}_i$:

$$\mathbf{p}_i \leftarrow \mathbf{q}_i. \quad (8.29)$$

3. **Sprawdzanie przeszkód na odcinku:** Jeśli oba punkty końcowe $\mathbf{p}_1$ i $\mathbf{p}_2$ są legalne, analizowana jest obecność przeszkód na odcinku L za pomocą funkcji $\mathcal{O}(L)$, która została opisana w sekcji 8.5:

$$\mathcal{O}(L) = \begin{cases} 1, & \text{jeśli na odcinku istnieje przeszkoda,} \\ 0, & \text{w przeciwnym razie.} \end{cases} \quad (8.30)$$

Jeśli $\mathcal{O}(L) = 1$, obliczany jest kąt nachylenia przeszkody θ względem osi x , definiowany jako:

$$\theta = \arctan \left(\frac{y_2 - y_1}{x_2 - x_1} \right). \quad (8.31)$$

4. **Przesunięcie punktów wzdłuż prostej:** Oba punkty końcowe $\mathbf{p}_1$ i $\mathbf{p}_2$ są przesuwane wzdłuż prostej wyznaczonej przez kąt θ , aż znajdą się w najbliższych legalnych pozycjach:

$$\mathbf{p}_i \leftarrow \mathbf{p}_i + \lambda \cdot (\cos \theta, \sin \theta), \quad (8.32)$$

gdzie λ jest minimalną odległością zapewniającą, że $\mathbf{p}_i$ znajdzie się w zbiorze punktów legalnych $\mathcal{M}$.

8.7.2 Implementacja algorytmu ICP z SVD

Algorytm ICP w proponowanej wersji wykorzystuje dekompozycję wartości osobliwych (SVD) do obliczania optymalnej transformacji między zbiorami punktów. Poniżej przedstawiono szczegóły implementacji.

Przyjęte podejście jest analogiczne do metody opisanej w pracy *Robust iterative closest point algorithm based on global reference point for rotation invariant registration* [91], gdzie zastosowano iteracyjny algorytm ICP oparty na globalnym punkcie referencyjnym. W szczególności, wykorzystanie dekompozycji SVD pozwala na wyznaczenie macierzy rotacji i translacji w sposób odporny na zakłócenia oraz zapewniający stabilność numeryczną. Dzięki temu możliwe jest precyzyjne dopasowanie zbiorów punktów nawet w obecności lokalnych błędów pomiarowych.

1. **Obliczanie centroidów:** Obliczane są centroidy obu zbiorów punktów – źródłowego (*source*) i docelowego (*target*):

$$\mathbf{c}_{\text{source}} = \frac{1}{N} \sum_{i=1}^N \mathbf{p}_{\text{source},i}, \quad \mathbf{c}_{\text{target}} = \frac{1}{N} \sum_{i=1}^N \mathbf{p}_{\text{target},i}. \quad (8.33)$$

2. **Obliczanie macierzy kowariancji:** Macierz kowariancji W jest obliczana jako:

$$W = \sum_{i=1}^N (\mathbf{p}_{\text{source},i} - \mathbf{c}_{\text{source}})(\mathbf{p}_{\text{target},i} - \mathbf{c}_{\text{target}})^T. \quad (8.34)$$

3. **Dekompozycja SVD:** Macierz W jest dekomponowana na macierze U , Σ i V :

$$W = U\Sigma V^T. \quad (8.35)$$

4. **Obliczanie rotacji i translacji:** Macierz rotacji $\mathbf{R}$ i wektor translacji $\mathbf{t}$ są obliczane jako:

$$\mathbf{R} = UV^T, \quad \mathbf{t} = \mathbf{c}_{\text{target}} - \mathbf{R}\mathbf{c}_{\text{source}}. \quad (8.36)$$

5. **Aktualizacja punktów:** Transformacja (rotacja i translacja) jest stosowana do punktów źródłowych, aktualizując ich pozycje.
6. **Sprawdzanie zbieżności:** Obliczany jest średni błąd między punktami źródłowymi a odpowiadającymi im punktami docelowymi. Jeśli błąd jest mniejszy od zadanego progu tolerancji, algorytm kończy działanie.

Jako następna lokalizacja użytkownika zwracany jest ostatni punkt na przekształconej ścieżce. Zaproponowane przemieszczenie jest następnie wykorzystywane jako jedno ze źródeł danych w filtrze Kalmana [50]. Drugim źródłem danych jest przemieszczenie określone za pomocą zmodyfikowanego filtra Madgwicka [92], a trzecim, jeśli dostępne, dane o pozycji użytkownika uzyskane z analizy obrazu kamery.

Rozdział 9

Sposób Użytkowania Aplikacji

9.1 Wprowadzenie

Aplikacja została zaprojektowana w sposób intuicyjny i przyjazny użytkownikowi, umożliwiając łatwe korzystanie z jej funkcji niezależnie od poziomu doświadczenia z technologią. Głównym celem było stworzenie rozwiązania, które będzie proste w obsłudze, a jednocześnie zaawansowane pod względem funkcjonalności, spełniając potrzeby zarówno nowych, jak i bardziej wymagających użytkowników.

Poniżej przedstawiono szczegółowy opis kroków, które użytkownik może podjąć podczas korzystania z aplikacji:

9.1.1 Wybór Budynku i Ustawienie Preferencji

Po uruchomieniu aplikacji użytkownik ma możliwość wyboru budynku, w którym chce się poruszać, z listy dostępnych obiektów. Lista ta może być filtrowana na podstawie lokalizacji GPS użytkownika, co dodatkowo przyspiesza proces wyboru. Po wskazaniu budynku użytkownik może przejść do ekranu ustawień, gdzie może skonfigurować swoje preferencje dotyczące trasy.

Dostępne opcje obejmują:

- wykluczenie schodów lub schodów ruchomych z wyznaczonej trasy,
- preferencję dla windy jako środka transportu między piętrami,
- zapisanie najczęściej wybieranych punktów docelowych jako ulubionych.

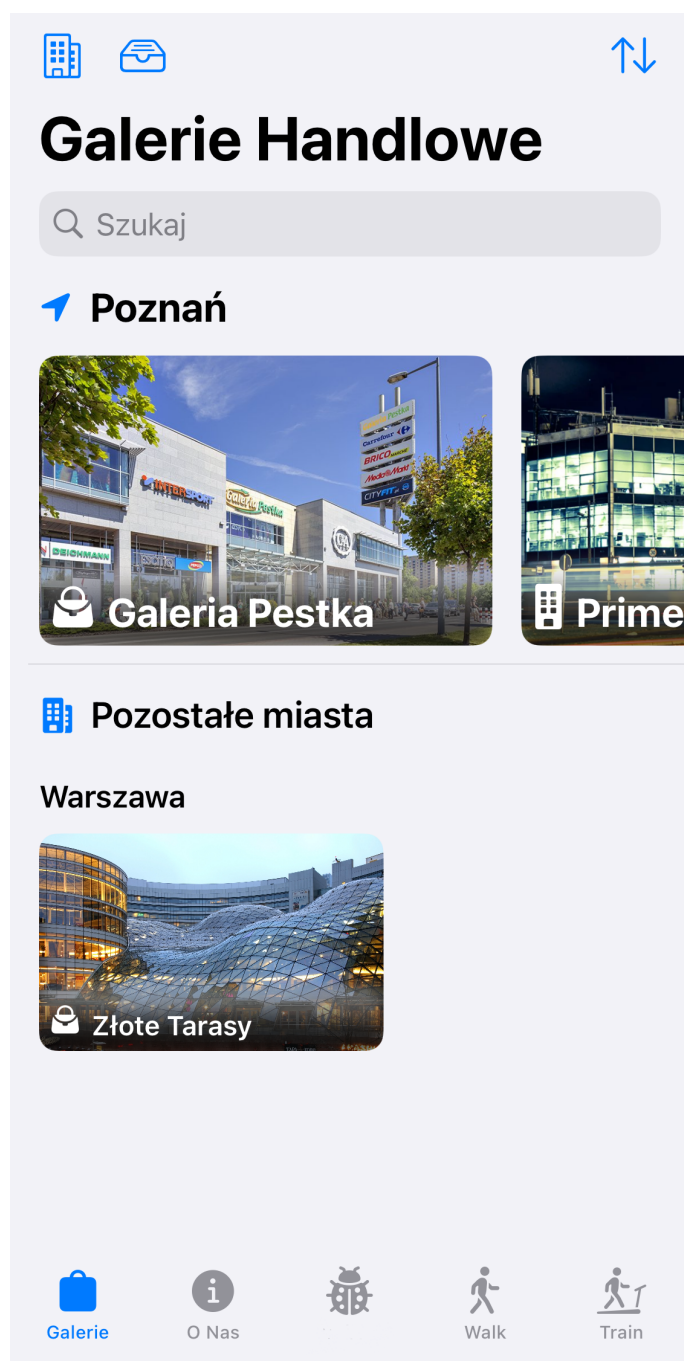
Po zapisaniu preferencji użytkownik przechodzi do kolejnego kroku, którym jest wybór konkretnego celu podróży.

9.1.2 Przeszukiwanie Listy Punktów Docelowych

Po wybraniu budynku, użytkownik może przeszukiwać listę dostępnych punktów docelowych w obrębie wybranego budynku. Użytkownik z listy dostępnych punktów docelowych może wybrać na przykład sklep, punkty informacyjny, toaletę z podziałem na płęć, windy czy schody. Po wybraniu celu aplikacja wskazuje optymalną trasę do celu.

9.1.3 Lokalizacja użytkownika oraz podążanie do celu

W celu precyzyjnego określenia pozycji użytkownika aplikacja wykorzystuje kamerę urządzenia do identyfikacji punktów charakterystycznych w otoczeniu, takich jak znaki informacyjne, wejścia



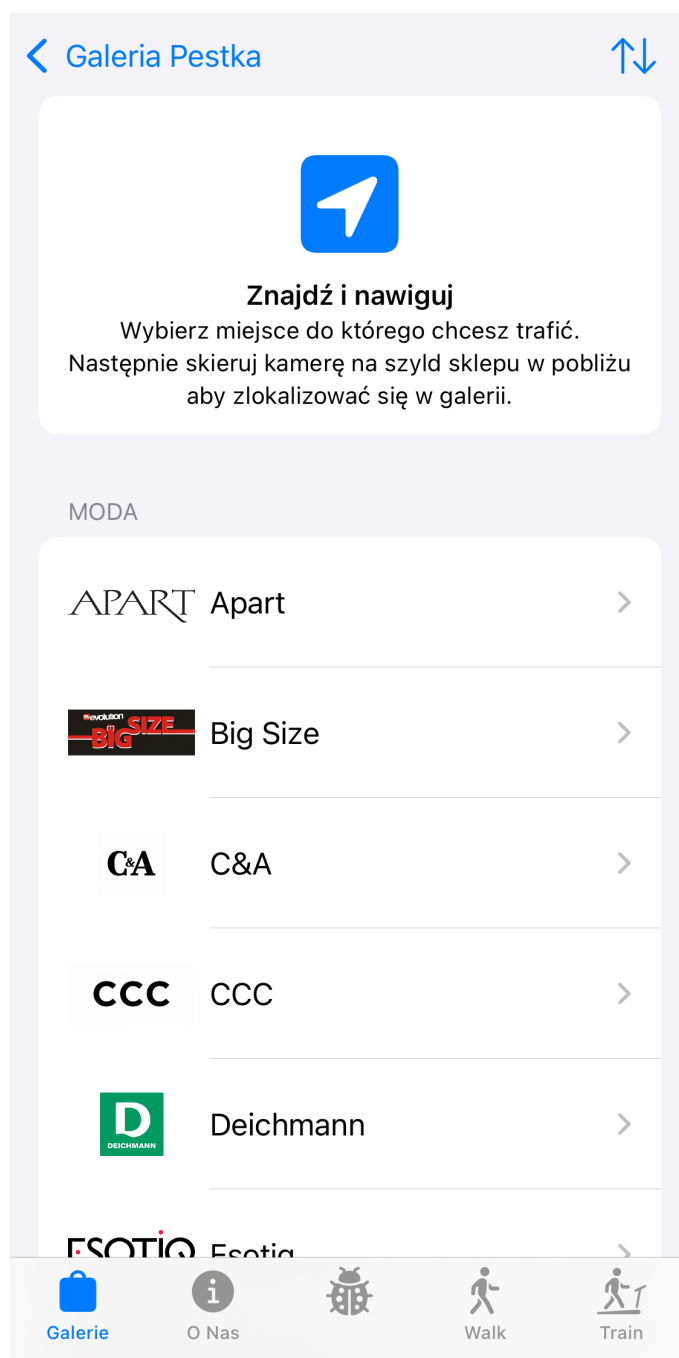
Rysunek 9.1

Strona główna aplikacji, umożliwiająca wybór budynku.

do sklepów czy inne punkty nawigacyjne. Proces ten jest wspomagany przez algorytmy sztucznej inteligencji, które analizują obraz w czasie rzeczywistym.

Po ustaleniu pozycji i wyznaczeniu trasy Ekran mapy przedstawia wizualizację planu wybranego budynku. Na mapie zaznaczone są punkty docelowe oraz dostępne trasy, którymi użytkownik może podążać. Interfejs pozwala na interakcję z mapą, umożliwiając użytkownikowi orientowanie, powiększanie oraz zmniejszanie widoku.

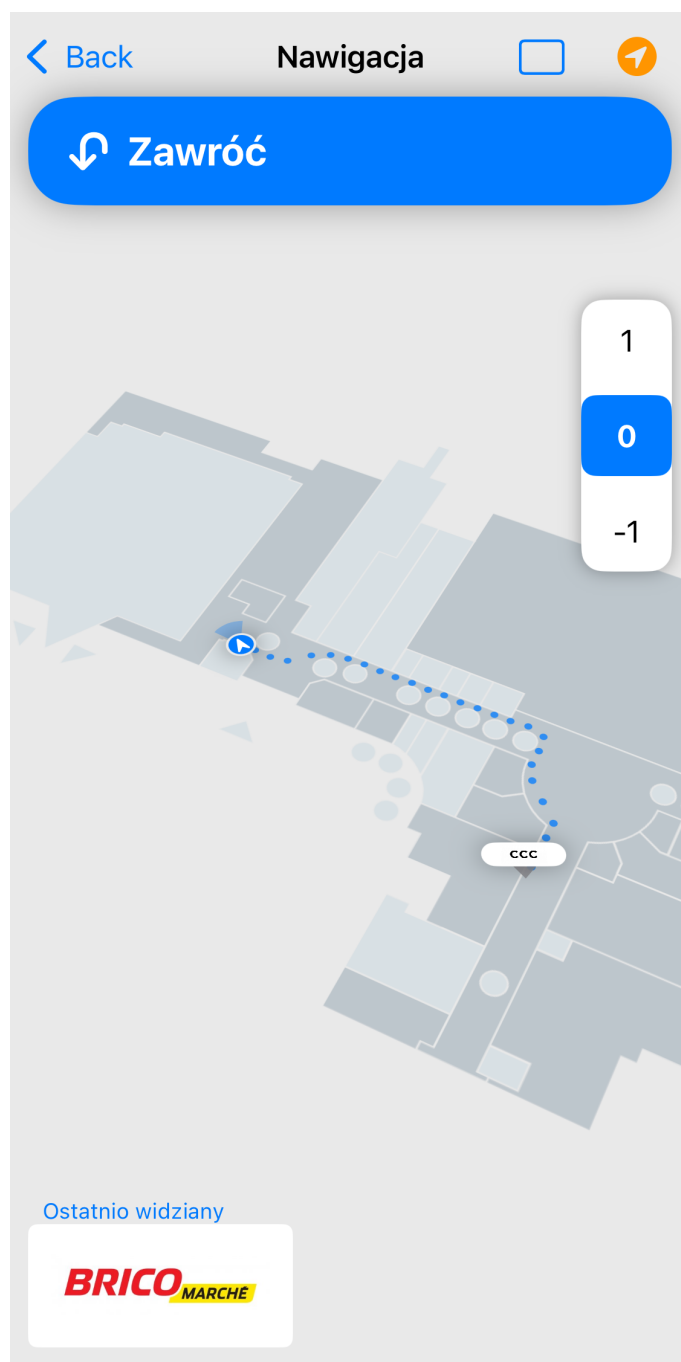
Aplikacja umożliwia użytkownikowi na trzymanie telefonu zarówno w pozycji pionowej, co pozwala na bieżące aktualizowanie pozycji przy pomocy kamery, jak i w pozycji poziomej, korzystając jedynie z danych czujników, co jest bardziej energooszczędne.



Rysunek 9.2

Ekran przeszukiwania punktów docelowych w budynku.

Dodatkowo aplikacja monitoruje postęp użytkownika na trasie i dostosowuje wskazówki w przypadku zboczenia z wyznaczonej ścieżki, oferując funkcję automatycznego przeliczania trasy. Cały proces został zaprojektowany z myślą o wygodzie użytkownika, uwzględniając różnorodne potrzeby i preferencje.



Rysunek 9.3
Wizualizacja mapy budynku z oznaczeniem punktów docelowych.

Rozdział 10

Zakończenie

Celem pracy było opracowanie wieloplatformowej aplikacji mobilnej do nawigacji wewnętrznej w budynkach, umożliwiającej dokładną lokalizację użytkownika oraz pomoc w dotarciu do wskazanego punktu docelowego. Zrealizowane założenia pozwoliły stworzyć aplikację, która działa zarówno na systemach Android, jak i iOS, zapewniając jej szeroką dostępność dla użytkowników urządzeń mobilnych.

Aplikacja umożliwia śledzenie przemieszczenia użytkownika w przestrzeni zamkniętej z dokładnością do 5 metrów oraz optymalizację trasy na podstawie danych z czujników wbudowanych w smartfony, takich jak akcelerometr, żyroskop oraz magnetometr. Dzięki tym funkcjom użytkownicy mogą łatwo nawigować po budynkach, korzystając z lokalizacji w czasie rzeczywistym oraz wizualizacji mapy, co zapewnia komfortowe i intuicyjne doświadczenie nawigacyjne.

Projekt koncentrował się na zapewnieniu pełnej funkcjonalności na różnych platformach, co umożliwiło przetestowanie aplikacji na różnych urządzeniach. Interfejs użytkownika został dostosowany do wymagań Androida i iOS, oferując płynne i wygodne użytkowanie na obu systemach operacyjnych.

Choć aplikacja spełnia swoje główne założenia, istnieją obszary, które mogą zostać rozwinięte w przyszłości. Warto rozważyć na przykład zastosowanie technologii takich jak Wi-Fi RTT czy BLE, które mogłyby dodatkowo zwiększyć precyzję lokalizacji. Można również rozwinąć system rekomendacji, który uwzględniałby potrzeby użytkowników, w tym osoby o ograniczonej mobilności. Integracja z systemami zarządzania budynkiem mogłaby pozwolić na dynamiczną aktualizację map w przypadku zmian w infrastrukturze, a także warto optymalizować aplikację pod kątem wydajności, szczególnie w dużych obiektach, aby zapewnić płynne działanie przy dużym natężeniu ruchu.

Podsumowując, opracowana aplikacja stanowi funkcjonalne rozwiązanie dla nawigacji wewnętrznej w budynkach, oferując precyzyjne śledzenie użytkownika i inteligentną optymalizację tras. Dzięki zaawansowanym technologiom i algorytmom, projekt stanowi solidną podstawę do dalszego rozwoju aplikacji, które mogą zrewolucjonizować sposób poruszania się w przestrzeniach zamkniętych.

Literatura

- [1] Tarun R. Katapally, Jasmin Bhawra, and Pinal Patel. A systematic review of the evolution of gps use in active living research: A state of the evidence for research, policy, and practice. *Health & Place*, 66, 2020.
- [2] Mikkel Baun Kjærgaard, Henrik Blunck, Torben Godsk, Thomas Toftkjær, Dan Lund Christensen, and Kaj Grønbaek. Indoor positioning using gps revisited. In Patrik Floréen, Antonio Krüger, and Mirjana Spasojevic, editors, *Pervasive Computing*, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [3] Victor Grinyak, A. Devyatisilny, and A. Shurygin. Feasibility study of indoor navigation with bluetooth beacons. *INFORMACIONNYE TEHNOLOGII*, 24, 2018.
- [4] Dimitrina Deliyka, Nikolay Yanev, and Mariana Trifonova. Methods for developing an indoor navigation system. *E3S Web of Conferences*, 280, 2021.
- [5] Rosen Ivanov. Indoor navigation system for visually impaired. *ACM International Conference Proceeding Series*, 471, 2010.
- [6] Dr S. Ar based indoor navigation system. *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 08, 2024.
- [7] Dawar Khan, Zhanglin Cheng, Hideaki Uchiyama, Sikandar Ali, Muhammad Asshad, and Kiyoshi Kiyokawa. Recent advances in vision-based indoor navigation: A systematic literature review. *Computers & Graphics*, 104, 2022.
- [8] Kai-Wei Chiang, Le Dinh Thuan, Trung Duong, and Rui Sun. The performance analysis of ins/gnss/v-slam integration scheme using smartphone sensors for land vehicle navigation applications in gnss-challenging environments. *Remote Sensing*, 12, 2020.
- [9] Alper Şen. Integration of landmarks extracted from human route descriptions using nlp into indoor navigation network model. In *Human route descriptions*, 2024.
- [10] Algan Yasar, Robert Voûte, and Edward Verbree. Direct use of indoor point clouds for path planning and navigation exploration in emergency situations. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLVIII-4/W11-2024, 2024.
- [11] Alessio Morganti, Fabio Grandi, Riccardo Khamaisi, and Margherita Peruzzini. *A Transdisciplinary Approach to Holographic Indoor Navigation Using Mixed Reality and Cloud Computing*. IOS Press, 2024.
- [12] Anirudh Maurya. Indoor navigation system. *International Journal for Research in Applied Science and Engineering Technology*, 12, 2024.
- [13] Satya Kiranmai Tadepalli, Preethivardhan Ega, and Pavan Inugurthi. Indoor navigation using augmented reality. *International Journal of Scientific Research in Science and Technology*, 2021.
- [14] Naser El-Sheimy and You Li. Indoor navigation: state of the art and future trends. *Satellite Navigation*, 2, 2021.

- [15] Jaiteg Singh, Noopur Tyagi, Saravjeet Singh, Farman Ali, and Daehan Kwak. A systematic review of contemporary indoor positioning systems: Taxonomy, techniques, and algorithms. *IEEE Internet of Things Journal*, 2024.
- [16] Heeket Mehta, Pratik Kanani, and Priya Lande. Google maps. *International Journal of Computer Applications*, 178, 2019.
- [17] Ján Hurtuk, Jakub Červeňák, Martin Štancel, Michal Hulič, and Peter Fecil'ak. Indoor navigation using indooratlas library. In *2019 IEEE 17th International Symposium on Intelligent Systems and Informatics (SISY)*, 2019.
- [18] Situm. Indoor positioning system. [Online; accessed 2025-01-11] <https://situm.com/en/technology/indoor-positioning-system/>.
- [19] Be My Eyes. About be my eyes. [Online; accessed 2025-01-11] <https://www.bemyeyes.com/about>.
- [20] John Spaulding, Alyssa Krauss, and Avinash Srinivasan. Exploring an open wifi detection vulnerability as a malware attack vector on ios devices. In *2012 7th International Conference on Malicious and Unwanted Software*, 2012.
- [21] Xin Yang, Haiyang Mei, Ke Xu, Xiaopeng Wei, Baocai Yin, and Rynson W. H. Lau. Where is my mirror? *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [22] Amanpreet Singh, Ronghang Hu, Vedanuj Goswami, Guillaume Couairon, Wojciech Galuba, Marcus Rohrbach, and Douwe Kiela. Flava: A foundational language and vision alignment model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [23] Lin Li, Eric Rigall, Junyu Dong, and Geng Chen. Mas3k: An open dataset for marine animal segmentation. In Felix Wolf and Wanling Gao, editors, *Benchmarking, Measuring, and Optimizing*, Cham, 2021. Springer International Publishing.
- [24] Xinyu Xiong, Zihuang Wu, Shuangyi Tan, Wenxue Li, Feilong Tang, Ying Chen, Siying Li, Jie Ma, and Guanbin Li. Sam2-unet: Segment anything 2 makes strong encoder for natural and medical image segmentation. *arXiv preprint arXiv:2408.08870*, 2024.
- [25] Papers with Code. Image segmentation - papers with code. [Online; accessed 2025-01-11] <https://paperswithcode.com/task/image-segmentation>.
- [26] Huankang Guan, Jiaying Lin, and Rynson W. H. Lau. Learning semantic associations for mirror detection. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [27] Ruozhen He, Jiaying Lin, and Rynson W.H. Lau. Efficient mirror detection via multi-level heterogeneous learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(1), 2023.
- [28] Marco Ortu, Giuseppe Destefanis, Bram Adams, Alessandro Murgia, Michele Marchesi, and Roberto Tonelli. The jira repository dataset: Understanding social aspects of software development. In *Proceedings of the 11th International Conference on Predictive Models and Data Analytics in Software Engineering*, PROMISE '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [29] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Fei-Fei Li. Imagenet: a large-scale hierarchical image database. *IEEE Conference on Computer Vision and Pattern Recognition*, 2009.
- [30] Andrei Barbu, David Mayo, Julian Alverio, William Luo, Christopher Wang, Dan Gutfreund, Joshua Tenenbaum, and Boris Katz. *ObjectNet: a large-scale bias-controlled dataset for pushing the limits of object recognition models*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [31] Li Deng. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Processing Magazine*, 29(6), 2012.

- [32] Papers with Code. Image classification - papers with code. [Online; accessed 2025-01-13] <https://paperswithcode.com/task/image-classification>.
- [33] Hugo Touvron, Andrea Vedaldi, Matthijs Douze, and Hervé Jégou. Fixing the train-test resolution discrepancy: Fixefficientnet. *ArXiv*, 2020.
- [34] Sachin Mehta and Mohammad Rastegari. Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer. In *International Conference on Learning Representations*, 2022.
- [35] Sachin Mehta, Farzad Abdolhosseini, and Mohammad Rastegari. Cvnets: High performance library for computer vision. In *Proceedings of the 30th ACM International Conference on Multimedia*, MM '22, 2022.
- [36] Qizhe Xie, Eduard H. Hovy, Minh-Thang Luong, and Quoc V. Le. Self-training with noisy student improves imagenet classification. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [37] Papers With Code. Real-time object detection - papers with code. [Online; accessed 2025-01-13] <https://paperswithcode.com/task/real-time-object-detection>.
- [38] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In David Fleet, Tomas Pajdla, Bernt Schiele, and Tinne Tuytelaars, editors, *Computer Vision – ECCV 2014*, Cham, 2014. Springer International Publishing.
- [39] Shihua Huang, Zhichao Lu, Xiaodong Cun, Yongjun Yu, Xiao Zhou, and Xi Shen. Deim: Detr with improved matching for fast convergence. *ArXiv*, 2024.
- [40] Momina Liaqat Ali and Zhou Zhang. The yolo framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers*, 13(12), 2024.
- [41] Yansong Peng, Hebei Li, Peixi Wu, Yueyi Zhang, and Xiaoyan Sun. D-fine: Redefine regression task in detrs as fine-grained distribution refinement. *ArXiv*, 2024.
- [42] Andreas Veit, Tomas Matera, Lukas Neumann, Jiri Matas, and Serge Belongie. Coco-text: Dataset and benchmark for text detection and recognition in natural images. *ArXiv*, 2016.
- [43] Shangbang Long, Siyang Qin, Dmitry Panteleev, Alessandro Bissacco, Yasuhisa Fujii, and Michalis Raptis. Towards end-to-end unified scene text detection and layout analysis. *ArXiv*, 2022.
- [44] Evgeny Myasnikov and Andrey Savchenko. Detection of sensitive textual information in user photo albums on mobile devices. In *2019 International Multi-Conference on Engineering, Computer and Information Sciences (SIBIRCON)*, 2019.
- [45] Darwin Bautista and Rowel Atienza. Scene text recognition with permuted autoregressive sequence models. In *European Conference on Computer Vision*, Cham, 2022. Springer Nature Switzerland.
- [46] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ArXiv*, abs/2010.11929, 2020.
- [47] Rishin Haldar and Debajyoti Mukhopadhyay. Levenshtein distance technique in dictionary lookup methods: An improved approach. *Computing Research Repository - CORR*, 2011.
- [48] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 1959.
- [49] Daniel Foad, Alifio Ghifari, Marchel Kusuma, Novita Hanafiah, and Eric Gunawan. A systematic literature review of a* pathfinding. *Procedia Computer Science*, 179, 2021.
- [50] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82(Series D), 1960.

- [51] Sebastian O. H. Madgwick. An efficient orientation filter for inertial and inertial / magnetic sensor arrays. *semanticscholar*, 2010.
- [52] CppReference. C++ 17 reference. [Online; accessed 2025-01-17]
<https://en.cppreference.com/w/cpp/17>.
- [53] Lutz Prechelt. Comparing java vs. c/c++ efficiency differences to interpersonal differences. *Communications of the ACM*, 42, 1999.
- [54] Yuwei Zhang. Comparative study of the execution efficiency of python and c++based on topological sorting. *Applied and Computational Engineering*, 34, 2024.
- [55] Vincent Ng. Rust vs c++, a battle of speed and efficiency. *Journal of Mathematical Techniques and Computational Mathematics*, 2023.
- [56] KimHao Nguyen, ThanhVu Nguyen, and Quoc-Sang Phan. Analyzing the cmake build system. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022.
- [57] Inc. Kitware. Cmake documentation. [Online; accessed 2025-01-14]
<https://cmake.org/documentation/>.
- [58] Google. Googletest. [Online; accessed 2024-01-16] <https://google.github.io/googletest/>.
- [59] Catch2 Team. Catch2 documentation. [Online; accessed 2024-01-16]
<https://catch2-temp.readthedocs.io/en/latest/index.html>.
- [60] IEEE Spectrum. Top programming languages 2024. [Online; accessed 2025-01-14]
<https://spectrum.ieee.org/top-programming-languages-2024>.
- [61] Kotlin Developers. Kotlin documentation. [Online; accessed 2025-01-14]
<https://kotlinlang.org/docs/home.html>.
- [62] Swift Developers. Swift documentation. [Online; accessed 2025-01-14]
<https://www.swift.org/documentation/>.
- [63] Djinni Developers. Djinni generator cli usage. [Online; accessed 2025-01-14]
<https://djinni.xlcpp.dev/djinni-generator/cli-usage/>.
- [64] ONNX Contributors. Onnx github repository. [Online; accessed 2025-01-18]
<https://github.com/onnx/onnx>.
- [65] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [66] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, and Xiaoqiang Zhang. Tensorflow: A system for large-scale machine learning. *ArXiv*, 2016.
- [67] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, Edouard Duchesnay, and Gilles Louppe. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2012.

- [68] Ferdin Joe John Joseph, Sarayut Nonsiri, and Annop Monsakul. *Keras and TensorFlow: A Hands-On Experience*. Springer International Publishing, Cham, 2021.
- [69] Microsoft. Onnx runtime github repository. [Online; accessed 2025-01-18]
<https://github.com/microsoft/onnxruntime>.
- [70] Federico Manca, Francesco Ratto, and Francesca Palumbo. Onnx-to-hardware design flow for adaptive neural-network inference on fpgas. In Luigi Carro, Francesco Regazzoni, and Christian Pilato, editors, *Embedded Computer Systems: Architectures, Modeling, and Simulation*, Cham, 2025. Springer Nature Switzerland.
- [71] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. Tensorflow-serving: Flexible, high-performance ml serving. *ArXiv*, 2017.
- [72] Marc Lalonde and Francis Charette Migneault. Contributing to libtorch: recent architectures and “vanilla” training pipeline. *Centre de recherche informatique de Montréal*, 2022.
- [73] Mohit Thakkar. *Beginning Machine Learning in iOS: CoreML Framework*. Springer, 2019.
- [74] Seok Kim, Jaewook Lee, Chang Kim, Lee Wonjun, and Seon Kim. Extending the onnx runtime framework for the processing-in-memory execution. In *2022 International Conference on Electronics, Information, and Communication (ICEIC)*, 2022.
- [75] Viswanathan Ramasamy, Vithya Ganesan, and A. Subramanian. *Graph Optimizations in Neural Networks by ONNX Model*. CRC Press, 2024.
- [76] Johan Barthélemy, Umair Iqbal, Yan Qian, Mehrdad Amirghasemi, and Pascal Perez. Safety after dark: A privacy compliant and real-time edge computing intelligent video analytics for safer public transportation. *Sensors*, 24, 2024.
- [77] R.A. Cherniavskyi and Yaroslav Krainyk. Usage of large language models for enhancing interactive systems: Challenges and prospects. *International Journal of Advanced Research*, 12, 2024.
- [78] Galeria Pestka. Plan centrum handlowego galeria pestka. [Online; accessed 2025-01-11]
<https://galeriapestka.pl/plan-centrum>.
- [79] Nooshin Hanafi and Hamid Saadatfar. A fast dbscan algorithm for big data based on efficient density calculation. *Expert Systems with Applications*, 203, 2022.
- [80] Franck Celestini and Fabrice Mortessagne. Cosine law at the atomic scale: Toward realistic simulations of knudsen diffusion. *Physical review. E, Statistical, nonlinear, and soft matter physics*, 77, 2008.
- [81] Marcos Henrique and Marcelo Bairral. Straight lines and angles that move, students’ ideas that touch and add. *Acta Scientiae*, 24, 2023.
- [82] George Grouios, Efthymios Ziagkas, Andreas Loukovitis, Konstantinos Chatzinikolaou, and Eirini Koidou. Accelerometers in our pocket: Does smartphone accelerometer technology provide accurate data? *Sensors*, 23, 2023.
- [83] Apple Inc. Cmpedometer documentation. [Online; accessed 2025-01-13]
<https://developer.apple.com/documentation/coremotion/cmpedometer>.
- [84] Google Inc. Read step count data - android developers. [Online; accessed 2025-01-13]
<https://developer.android.com/health-and-fitness/guides/basic-fitness-app/read-step-count-data>.
- [85] Minghui Du, Bo Liang, He Wang, Peng Xu, Ziren Luo, and Yueliang Wu. Advancing space-based gravitational wave astronomy: Rapid parameter estimation via normalizing flows. *Science China Physics, Mechanics & Astronomy*, 67, 2024.

- [86] Michael Hölzl, Roland Neumeier, and Gerald Ostermayer. Analysis of compass sensor accuracy on several mobile devices in an industrial environment. In *Lecture Notes in Computer Science*. Springer, 2013.
- [87] Shota Vashakmadze. Modeling the line: Bresenham’s algorithm, 1962–87. *Architectural Theory Review*, 24, 2021.
- [88] Apple Developer Documentation. Clocation: altitude. [Online; accessed 2025-01-14]
<https://developer.apple.com/documentation/corelocation/cllocation/altitude>.
- [89] K.S. Arun, T.S. Huang, and Steven Blostein. Least-squares fitting of two 3-d point sets. *iee t pattern anal. Pattern Analysis and Machine Intelligence, IEEE Transactions on*, PAMI-9, 1987.
- [90] Fang Wang and Zijian Zhao. A survey of iterative closest point algorithm. In *2017 Chinese Automation Congress (CAC)*. IEEE, 2017.
- [91] Shaoyi Du, Yiting Xu, Teng Wan, Huaizhong Hu, Sirui Zhang, Guanglin Xu, and Xuetao Zhang. Robust iterative closest point algorithm based on global reference point for rotation invariant registration. *PLOS ONE*, 12, 2017.
- [92] Amjed Al-Fahoum and Momtaz Abadir. Design of a modified madgwick filter for quaternion-based orientation estimation using ahrs. *International Journal of Computer Electrical Engineering*, 10, 2018.



© 2025 Mikołaj Nowak

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.